WNDRVR
SUPPORT NETWORK

API REFERENCE: KERNEL SECURITY, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

API Reference: Kernel Security, SR0650

20 July 2020

1. DISKENCRYPTIONTOOLS

Name

diskEncryptionTools - Disk encryption tools

Layer

DISK_ENCRYPTION

Routines

diskEncrypt( ) - encrypt an entire disk partition **diskDecrypt( )** - decrypt an entire encrypted disk partition **diskMountCrypto( )** - mount an encrypted disk partition

Description

Provide a set of APIs for encrypting, decrypting and mount a disk partition.

You must create a secrets store and import a valid key into the secrets store before call the APIs, otherwise the operations will fail. For information about the secret store and encryption keys, see the Wind River Cryptography Libraries for VxWorks 7 Programmer's Guide.

A typical usage of the APIs is shown below. It shows how to encrypt "/ata0:1".

```
secSecretImport ("0", "vxworks", 8);
diskEncrypt ("/ata0:1", "0");
```

Include files

diskEncryptionTools.h

Name

diskEncrypt( ) - encrypt an entire disk partition

Layer

DISK_ENCRYPTION

Synopsis

```
STATUS diskEncrypt
    (
    char * pDevName,
    char * pKeyId
    )
```

Description

This function encrypts an entire disk partition. Before encrypting, be sure that a valid file system has been instantiated on the partition. (dosFs or HRFS). Do not attempt to halt encryption while it is in process. Doing so will corrupt the data in the partition.

Arguments

pDevName

pointer to the partition name

pKeyId

the key ID associated with the partition (which is used to fetch the master key from the secrets store).

Returns

OK if the operation was successfully performed, **ERROR** if it failed.

Errno

N/A

See also

diskEncryptionTools

Name

diskDecrypt( ) - decrypt an entire encrypted disk partition

Layer

DISK_ENCRYPTION

Synopsis

```
STATUS diskDecrypt
    (
    char * pDevName,
    char * pKeyId
    )
```

Description

This function decrypts an entire encrypted disk partition. Be sure that the key ID and associated key are correct before calling the routine. Otherwise, the data on the partition will be corrupted. Do not attempt to halt decryption while it is in process. Doing so will corrupt the data in the partition.

Arguments

pDevName

pointer to the partition name

pKeyId

the key ID associated with the partition (which is used to fetch the master key from the secrets store).

Returns

OK if the operation was successfully performed, **ERROR** if it failed.

Errno

N/A

See also

diskEncryptionTools

Name

diskMountCrypto( ) - mount an encrypted disk partition

Layer

DISK_ENCRYPTION

Synopsis

```
STATUS diskMountCrypto
    (
    char * pDevName,
    char * pKeyId
    )
```

Description

This function mounts an encrypted disk partition. If you have not enabled auto-mounting, or if a mount operation has failed, you can call this routine to mount an encrypted partition at runtime (programmatically or from the kernel shell).

Arguments

pDevName

pointer to the partition name

pKeyId

the key ID associated with the partition (which is used to fetch the master key from the secrets store).

Returns

OK if the operation was successfully performed, **ERROR** if it failed.

Errno

N/A

See also

diskEncryptionTools

2. IPFREESCALE

Name

ipfreescale - implementation of hardware cryptographic support for Freescale SEC 2 processor

Layer

CRYPTOMISC_IPFREESCALE

Routines

ipfreescale_async_done_cbf_register( ) - Register async job done call back routine **ipfreescale_async_error_cbf_register( )** - Register async job error call back routine **ipfreescale_md5_digest( )** - Calculate message digest using the MD5 algorithm. **ipfreescale_md5_hmac( )** - Calculate HMAC using the MD5 algorithm. **ipfreescale_md5_digest_init( )** - initialize MD5 message digest context **ipfreescale_md5_digest_update( )** - update MD5 message digest context **ipfreescale_md5_digest_final( )** - finalize MD5 message digest context **ipfreescale_sha1_digest( )** - calculate message digest using the SHA1 algorithm **ipfreescale_sha1_hmac( )** - Calculate HMAC using the SHA1 algorithm. **ipfreescale_sha1_digest_init( )** - initialize SHA1 message digest context **ipfreescale_sha1_digest_update( )** - update SHA1 message digest context **ipfreescale_sha1_digest_final( )** - Finalize SHA1 message digest context **ipfreescale_digest_copy( )** - copy message digest context **ipfreescale_digest_cleanup( )** - clean up message digest context **ipfreescale_des_cbc_cipher( )** - encrypt or decrypt using DES-CBC cipher **ipfreescale_des_ecb_cipher( )** - encrypt or decrypt using DES-ECB cipher **ipfreescale_3des_cbc_cipher( )** - encrypt or decrypt using 3DES-CBC cipher **ipfreescale_3des_ecb_cipher( )** - encrypt or decrypt using 3DES-ECB cipher **ipfreescale_aes128_cbc_cipher( )** - encrypt or decrypt using the AES-CBC-128 cipher **ipfreescale_aes128_ecb_cipher( )** - encrypt or decrypt using the AES-ECB-128 cipher **ipfreescale_aes192_cbc_cipher( )** - encrypt or decrypt using the AES-CBC-192 cipher **ipfreescale_aes192_ecb_cipher( )** - encrypt or decrypt using the AES-ECB-192 cipher **ipfreescale_aes256_cbc_cipher( )** - encrypt or decrypt using the AES-CBC-256 cipher **ipfreescale_aes256_ecb_cipher( )** - encrypt or decrypt using the AES-ECB-256 cipher **ipfreescale_rc4_cipher( )** - RC4 stream cipher **ipfreescale_mod_exp( )** - modular exponentiation **ipfreescale_random_gen( )** - generate sequence of random data **ipfreescale_ccmp_encrypt( )** - Counter mode with Cipher Block chaining **ipfreescale_aesccm( )** - AES Counter mode with Cipher Block chaining **ipfreescale_crc( )** - calculate the CRC checksum on data **ipfreescale_dma( )** - DMA copy data using freescale engine **ipfreescale_ipsec_esp_des_cbc_md5_input( )** - IPsec ESP input processing using DES-CBC-MD5 **ipfreescale_ipsec_esp_des_cbc_md5_output( )** - IPsec ESP output processing using DES-CBC-MD5 **ipfreescale_ipsec_esp_des_cbc_sha1_input( )** - IPsec ESP input processing using DES-CBC-SHA1 **ipfreescale_ipsec_esp_des_cbc_sha1_output( )** - IPsec ESP output processing using DES-CBC-SHA1 **ipfreescale_ipsec_esp_3des_cbc_md5_input( )** - IPsec ESP input processing using 3DES-CBC-MD5 **ipfreescale_ipsec_esp_3des_cbc_md5_output( )** - IPsec ESP output processing using 3DES-CBC-MD5 **ipfreescale_ipsec_esp_3des_cbc_sha1_input( )** - IPsec ESP input processing using 3DES-CBC-SHA1 **ipfreescale_ipsec_esp_3des_cbc_sha1_output( )** - IPsec ESP output processing using 3DES-CBC-SHA1 **ipfreescale_ipsec_esp_aes_cbc_md5_input( )** - IPsec ESP input processing using AES-CBC-MD5 **ipfreescale_ipsec_esp_aes_cbc_md5_output( )** - IPsec ESP output processing using AES-CBC-MD5 **ipfreescale_ipsec_esp_aes_cbc_sha1_input( )** - IPsec ESP input processing using AES-CBC-SHA1 **ipfreescale_ipsec_esp_aes_cbc_sha1_output( )** - IPsec ESP output processing using AES-CBC-SHA1

Description

This library provides an implementation of a cryptographic engine for the Freescale SEC 2 and SEC 4 processor, for use with Wind River Cryptography Libraries.

Include files

none

Name

ipfreescale_async_done_cbf_register( ) - Register async job done call back routine

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_async_done_cbf_register
    (
    void (*cbf)(void *)
    );
```

Description

This routine registers the call back routine for successfull completed non-IPsec asynchronous jobs. It must be called before submitting an asynchronous job. The call back routine need o have a (void *) handle as input parameter to identify the completed job.

Parameters:

cbf

Pointer to call back function.

Returns

1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_async_error_cbf_register( ) - Register async job error call back routine

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_async_error_cbf_register
    (
    void (*cbf)(void *, int)
    );
```


WNDRVR

Description

This routine registers the call back routine for completed non-IPsec asynchronous jobs with errors. It must be called before submitting an asynchronous job. The call back routine need o have a (void *) handle to identify the completed job and an (int) status as input parameters.

Parameters:

cbf

Pointer to call back function.

Returns

1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_md5_digest( ) - Calculate message digest using the MD5 algorithm.

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_md5_digest
    (
    unsigned char *data,
    unsigned int  length,
    unsigned char *digest
    );
```

Description

This routine calculates a message digest using the MD5 algorithm.

Parameters:

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

digest

Pointer to buffer for the digest (16 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_md5_hmac( ) - Calculate HMAC using the MD5 algorithm.

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_md5_hmac
    (
    unsigned char *data,
    unsigned int  length,
    unsigned char *key,
    unsigned int  keylength,
    unsigned char *hmac
    );
```

Description

This routine calculates a HMAC using the MD5 algorithm.

Parameters:

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

key

Pointer to buffer with the key.

keylength

Length of key in bytes.

hmac

Pointer to buffer for the hmac (16 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_md5_digest_init( ) - initialize MD5 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_md5_digest_init
    (
    void *ctx
    );
```

Description

This routine initializes an MD5 message context.

Parameters:

ctx

Pointer to the context.

Returns

1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_md5_digest_update( ) - update MD5 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_md5_digest_update
    (
    void                *ctx,
    const unsigned char *data,
    unsigned int        length
    );
```

Description

This routine updates an MD5 message context.

Parameters:

ctx

Pointer to the context.

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

Returns

-1 = error, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_md5_digest_final( ) - finalize MD5 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_md5_digest_final
    (
    void          *ctx,
    unsigned char *digest
    );
```

Description

This routine finalizes an MD5 message digest context.

Parameters:

ctx

Pointer to the context.

digest

Pointer to buffer for the digest (16 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_sha1_digest( ) - calculate message digest using the SHA1 algorithm

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_sha1_digest
    (
    unsigned char *data,
    unsigned int  length,
    unsigned char *digest
    );
```

Description

This routine calculates a message digest using the SHA1 algorithm.

Parameters:

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

digest

Pointer to buffer for the digest (20 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_sha1_hmac( ) - Calculate HMAC using the SHA1 algorithm.

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_sha1_hmac
    (
    unsigned char *data,
    unsigned int  length,
    unsigned char *key,
    unsigned int  keylength,
    unsigned char *hmac
    );
```

Description

This routine calculates a HMAC using the SHA1 algorithm.

Parameters:

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

key

Pointer to buffer with the key.

keylength

Length of key in bytes.

hmac

Pointer to buffer for the hmac (20 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_sha1_digest_init( ) - initialize SHA1 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_sha1_digest_init
    (
    void *ctx
    );
```

Description

This routine initializes a SHA1 message digest context.

Parameters:

ctx

Pointer to the context.

Returns

1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_sha1_digest_update( ) - update SHA1 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_sha1_digest_update
    (
    void                *ctx,
    const unsigned char *data,
    unsigned int        length
    );
```

Description

This routine updates a SHA1 message digest context.

Parameters:

ctx

Pointer to the context.

data

Pointer to buffer with the data to hash.

length

Length of data in bytes.

Returns

-1 = error, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_sha1_digest_final( ) - Finalize SHA1 message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_sha1_digest_final
    (
    void          *ctx,
    unsigned char *digest
    );
```

Description

This routine finalizes a SHA1 message digest context.

Parameters:

ctx

Pointer to the context.

digest

Pointer to buffer for the digest (20 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_digest_copy( ) - copy message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_digest_copy
    (
```

```
    void       *ctx_to,
    const void *ctx_from
    );
```

Description

This routine copies a message digest context.

Parameters:

ctx_to

Pointer to the destination context.

ctx_from

Pointer to the source context.

Returns

-1 = error, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_digest_cleanup( ) - clean up message digest context

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_digest_cleanup
    (
    void *ctx
    );
```

Description

This routine cleans up a message digest context.

Parameters:

ctx

Pointer to the context.

Returns

1= ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_des_cbc_cipher( ) - encrypt or decrypt using DES-CBC cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_des_cbc_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned char       *iv,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the DES-CBC cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (8 bytes).

iv

Initialization vector (8 bytes). Updated by hardware.

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_des_ecb_cipher( ) - encrypt or decrypt using DES-ECB cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_des_ecb_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the DES-ECB cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (8 bytes).

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_3des_cbc_cipher( ) - encrypt or decrypt using 3DES-CBC cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_3des_cbc_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned char       *iv,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the 3DES-CBC cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (24 bytes).

iv

Initialization vector (8 bytes). Updated by hardware.

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_3des_ecb_cipher( ) - encrypt or decrypt using 3DES-ECB cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_3des_ecb_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the 3DES-ECB cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (24 bytes).

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes128_cbc_cipher( ) - encrypt or decrypt using the AES-CBC-128 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes128_cbc_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned char       *iv,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-CBC-128 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (16 bytes).

iv

Initialization vector (16 bytes). Updated by hardware.

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes128_ecb_cipher( ) - encrypt or decrypt using the AES-ECB-128 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes128_ecb_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-ECB-128 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (16 bytes).

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes192_cbc_cipher( ) - encrypt or decrypt using the AES-CBC-192 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes192_cbc_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned char       *iv,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-CBC-192 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (24 bytes).

iv

Initialization vector (24 bytes). Updated by hardware.

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes192_ecb_cipher( ) - encrypt or decrypt using the AES-ECB-192 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes192_ecb_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-ECB-192 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (24 bytes).

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes256_cbc_cipher( ) - encrypt or decrypt using the AES-CBC-256 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes256_cbc_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned char       *iv,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-CBC-256 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (32 bytes).

iv

Initialization vector (32 bytes). Updated by hardware.

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aes256_ecb_cipher( ) - encrypt or decrypt using the AES-ECB-256 cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aes256_ecb_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    int                 enc
    );
```

Description

This routine encrypts or decrypts a data buffer using the AES-ECB-256 cipher.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key (32 bytes).

enc

1 = encryption. 0 = decryption.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_rc4_cipher( ) - RC4 stream cipher

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_rc4_cipher
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    void                *key,
    unsigned int        keylen,
    unsigned char       *ctxi,
    unsigned char       *ctxo
    );
```

Description

This routine performs an RC4 stream cipher. Provide key and output context for operation with key load and context save. Provide input context and output context for operation with context load and context save.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt/decrypt.

length

Length of data in bytes.

key

Pointer to buffer with the key.

keylen

Length of encryption key (1-16 bytes).

ctxi

Buffer with input context (259 bytes).

ctxo

Buffer with output context (259 bytes).

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_mod_exp( ) - modular exponentiation

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_mod_exp
    (
    unsigned char       *out,
    const unsigned char *base,
    const unsigned char *e,
    const unsigned char *mod,
```

```
    unsigned int        modlen
    );
```

Description

This routine performs modular exponentiation. Calculates b^e mod m. The size of the base and exponent buffers are assumed to be of equal size as the modulus length and zero padded with leading zeros if necessary. The size of the result buffer is also equal to the modulus length and the result will be zero padded if necessary.

Parameters:

out

Pointer to the result.

base

Pointer to buffer with the base.

e

Pointer to buffer with the exponent.

mod

Pointer to buffer with the result.

modlen

Length of modulus.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_random_gen( ) - generate sequence of random data

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_random_gen
    (
    unsigned char *data,
```

```
    unsigned int  length
    );
```

Description

This routine generates a sequence of random data.

Parameters:

data

Pointer to buffer for the generated random data.

length

Number of bytes in the sequence.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ccmp_encrypt( ) - Counter mode with Cipher Block chaining

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_ccmp_encrypt
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    unsigned char       *mic,
    const unsigned char *aad,
    unsigned int        aadlen,
    void                *key,
    unsigned int        keylen,
    unsigned char       *ctxi
    );
```

Description

This routine performs CCMP encryption. (AES-CCM according to IEEE802.11) Provide key and output context for operation with key load and context save. Provide input context for operation with context load.

Parameters:

out

Pointer to the result.

in

Pointer to buffer with the data to encrypt.

length

Length of data in bytes.

mic

Pointer to Message Integrity Code

aad

Pointer to Additional Authentication Data

aadlen

Additional Authentication Data Length

key

Pointer to buffer with the key.

keylen

Length of encryption key.

ctxi

Buffer with input context.

Returns

-1 = error, 0 = timeout, 1 = ok

Errno

N/A

See also

ipfreescale

Name

ipfreescale_aesccm( ) - AES Counter mode with Cipher Block chaining

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_aesccm
    (
    unsigned char       *out,
    const unsigned char *in,
    unsigned int        length,
    unsigned char       *micout,
    unsigned char       *micin,
    unsigned int        miclen,
    const unsigned char *aad,
    unsigned int        aadlen,
    void                *key,
    unsigned int        keylen,
    unsigned char       *ctxi,
    int                 enc,
    int                 async,
    void *              asynchandle,
    unsigned int        options);
```

Description

This routine performs AES - CCM encryption or decryption with associated generation or verification of authenticity. Provide key and output context for operation with key load and context save. Provide input context for operation with context load. Failed integrity check during decrypt will optionally generate an error.

Parameters:

out

Pointer to the buffer or scatter gather table (if options **IPFREESCALE_OPT_OUTPUTDATA_IS_SGT** or **IPFREESCALE_OPT_OUTPUTDATA_IS_HW_SGT** are set) for output data.

in

Pointer to buffer or scatter gather table (if options **IPFREESCALE_OPT_INPUTDATA_IS_SGT** or **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT** are set) with the input data to process.

length

Length of in data in bytes.

micout

Pointer to output Message Integrity Code

micin

Pointer to input Message Integrity Code

miclen

Length in bytes of Message Integrity Code

aad

Pointer to Additional Authentication Data Set to **NULL** if there isn't any AAD.

aadlen

Additional Authentication Data Length Set to 0 if there isn't any AAD

key

Pointer to buffer with the key.

keylen

Length in bytes of encryption key.

ctxi

Buffer with input context.

enc

1 for encrypt, 0 for decrypt.

async

TRUE for asynchronous operation, **FALSE** for synchronous operation

asynchandle

Handle to identify the job in case of asynchronous job. Should be set to **NULL** for synchronous jobs.

options

Ored bit options to control AES CCM. Valid options: **IPFREESCALE_OPT_NONE**
IPFREESCALE_OPT_AESCCM_ICV **IPFREESCALE_OPT_INPUTDATA_IS_SGT**
IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT **IPFREESCALE_OPT_OUTPUTDATA_IS_SGT**
IPFREESCALE_OPT_OUTPUTDATA_IS_HW_SGT

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_crc( ) - calculate the CRC checksum on data

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_crc
    (
    const unsigned char *indata,
    unsigned int        length,
    unsigned char       *crc,
    unsigned int        polynomial,
    unsigned int        inctx,
    int                 async,
    void *              asynchandle,
    unsigned int        options
    );
```

Description

This routine generates CRC checksum on supplied data.

Parameters:

polynomial

CRC polynomial. E.g. for IEEE802: **IPFREESCALE_CRC32_POLY_IEEE802**

indata

Pointer to buffer for data to generate CRC checksum from. Or pointer to scatter gather table if options **IPFREESCALE_OPT_INPUTDATA_IS_SGT** or **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT** are set.

length

Number of bytes in the sequence. If indata is a SGT, total length of valid data.

inctx

Input context data (initial CRC value).

async

TRUE for asynchronous operation, **FALSE** for synchronous operation

asynchandle

Handle to identify the job in case of asynchronous job. Should be set to **NULL** for synchronous jobs.

options

Ored bit options to control CRC generation. Valid options: **IPFREESCALE_OPT_NONE**
IPFREESCALE_OPT_INPUTDATA_IS_SGT **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT**
IPFREESCALE_OPT_CRC_IEEE802 **IPFREESCALE_OPT_CRC_IETF3385**
IPFREESCALE_OPT_CRC_DIS **IPFREESCALE_OPT_CRC_DOS**
IPFREESCALE_OPT_CRC_DOC

crc

Pointer to where to store the calculated CRC.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_dma( ) - DMA copy data using freescale engine

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
int ipfreescale_dma
    (
    const unsigned char *indata,
    unsigned int        length,
    unsigned char       *outdata,
    int                 async,
    void *              asynchandle,
    unsigned int        options
    );
```

Description

This routine copies data using the freescale engine.

Parameters:

indata

Pointer to buffer for data to be copied. Or pointer to scatter gather table if options **IPFREESCALE_OPT_INPUTDATA_IS_SGT** or **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT** are set.

length

Number of bytes in the sequence. If indata is a SGT, this is the total length of valid data.

async

TRUE for asynchronous operation, **FALSE** for synchronous operation

asynchandle

Handle to identify the job in case of asynchronous job. Should be set to **NULL** for synchronous jobs.

options

Ored bit options to control CRC generation. Valid options: **IPFREESCALE_OPT_NONE** **IPFREESCALE_OPT_INPUTDATA_IS_SGT** **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT** **IPFREESCALE_OPT_OUTPUTDATA_IS_SGT** **IPFREESCALE_OPT_OUTPUTDATA_IS_HW_SGT**

outdata

Pointer to where to copy the data. Or pointer to scatter gather table if options **IPFREESCALE_OPT_OUTPUTDATA_IS_SGT** or **IPFREESCALE_OPT_INPUTDATA_IS_HW_SGT** are set.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_des_cbc_md5_input( ) - IPsec ESP input processing using DES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_des_cbc_md5_input
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP input processing using DES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_des_cbc_md5_output( ) - IPsec ESP output processing using DES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_des_cbc_md5_output
    (
    UINT8                  *data,
    UINT32                 length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                  *digest,
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs IPsec ESP output processing using DES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_des_cbc_sha1_input( ) - IPsec ESP input processing using DES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_des_cbc_sha1_input
    (
    UINT8                  *data,
    UINT32                 length,
    Iphwcrypto_esp_cbc_sa  *sa,
    UINT8                  *digest,
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs input processing using DES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_des_cbc_sha1_output( ) - IPsec ESP output processing using DES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_des_cbc_sha1_output
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP output processing using DES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2= kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_3des_cbc_md5_input( ) - IPsec ESP input processing using 3DES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_3des_cbc_md5_input
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP input processing using 3DES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_3des_cbc_md5_output( ) - IPsec ESP output processing using 3DES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_3des_cbc_md5_output
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
```

```
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs IPsec ESP output processing using 3DES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_3des_cbc_sha1_input( ) - IPsec ESP input processing using 3DES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_3des_cbc_sha1_input
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP input processing using 3DES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_3des_cbc_sha1_output( ) - IPsec ESP output processing using 3DES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_3des_cbc_sha1_output
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP output processing using 3DES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_aes_cbc_md5_input( ) - IPsec ESP input processing using AES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_aes_cbc_md5_input
    (
    UINT8                 *data,
    UINT32                length,
    Iphwcrypto_esp_cbc_sa *sa,
    UINT8                 *digest,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt
    );
```

Description

This routine performs IPsec ESP input processing using AES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_aes_cbc_md5_output( ) - IPsec ESP output processing using AES-CBC-MD5

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_aes_cbc_md5_output
    (
    UINT8                  *data,
    UINT32                 length,
    Iphwcrypto_esp_cbc_sa  *sa,
    UINT8                  *digest,
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs IPsec ESP output processing using AES-CBC-MD5.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_aes_cbc_sha1_input( ) - IPsec ESP input processing using AES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_aes_cbc_sha1_input
    (
    UINT8                  *data,
    UINT32                 length,
    Iphwcrypto_esp_cbc_sa  *sa,
    UINT8                  *digest,
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs IPsec ESP input processing using AES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

Name

ipfreescale_ipsec_esp_aes_cbc_sha1_output( ) - IPsec ESP output processing using AES-CBC-SHA1

Layer

CRYPTOMISC_IPFREESCALE

Synopsis

```
IP_PUBLIC int ipfreescale_ipsec_esp_aes_cbc_sha1_output
    (
    UINT8                  *data,
    UINT32                 length,
    Iphwcrypto_esp_cbc_sa  *sa,
    UINT8                  *digest,
    Ipipsec_param          *param,
    Ipcom_pkt              *pkt
    );
```

Description

This routine performs IPsec ESP output processing using AES-CBC-SHA1.

Parameters:

data

Pointer to the packet data.

length

Length of the packet.

sa

Pointer to the sa (Security Association).

digest

Pointer to the message authentication code.

param

Pointer to IPsec Parameters

pkt

Pointer to the packet structure.

Returns

-1 = error, 0 = timeout, 1 = ok, 2 = kept

Errno

N/A

See also

ipfreescale

3. IPHWCRYPTO

Name

iphwcrypto - public API of cyptographic hardware support for Wind River Cryptography Libraries

Layer

CRYPTOMISC_IPHWCRYPTO

Routines

iphwcrypto_engine_add( ) - add an engine to the system **iphwcrypto_engine_remove( )** - remove an engine from the system **iphwcrypto_engine_add_digest( )** - add a digest method to an engine **iphwcrypto_engine_register_digests( )** - register added digest methods **iphwcrypto_engine_add_cbc_cipher( )** - add a CBC cipher method to an engine **iphwcrypto_engine_add_ecb_cipher( )** - add an ECB cipher method to an engine **iphwcrypto_engine_register_ciphers( )** - register added cipher methods **iphwcrypto_engine_add_random( )** - add random number generation method to an engine **iphwcrypto_engine_register_random( )** - register added random number generator **iphwcrypto_engine_add_dh( )** - add Diffie-Hellman method to an engine **iphwcrypto_engine_register_dh( )** - register added Diffie-Hellman method **iphwcrypto_engine_add_rsa( )** - add RSA method to an engine **iphwcrypto_engine_register_rsa( )** - register added RSA method **iphwcrypto_engine_add_dsa( )** - add DSA method to an engine **iphwcrypto_engine_register_dsa( )** - register added DSA method **iphwcrypto_ipsec_engine_add_esp( )** - add IPsec ESP method to an engine **iphwcrypto_ipsec_engine_register_esp( )** - register added IPsec ESP method **iphwcrypto_ipsec_esp_re_input( )** - re-input IPsec ESP packet **iphwcrypto_ipsec_esp_re_output( )** - re-output IPsec ESP packet **iphwcrypto_ipsec_esp_pkt_free( )** - return previously kept packet to the system

Description

This library contains the API for cryptographic hardware support for Wind River Cryptographic Libraries.

Include files

none

Name

iphwcrypto_engine_add( ) - add an engine to the system

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add
    (
    const char *id,
    const char *name
    );
```

Description

This routine adds a hardware cryptography engine to the system.

Parameters:

id

The id of the engine.

name

The name of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_NO_MEMORY** = out of memory, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_remove( ) - remove an engine from the system

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_remove
    (
    const char *id
    );
```

Description

This routine removes an engine from the system.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_digest( ) - add a digest method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_digest
    (
    const char                *id,
    int                       type,
    int                       ctx_size,
    Iphwcrypto_digest_init    init,
    Iphwcrypto_digest_update  update,
    Iphwcrypto_digest_final   final,
    Iphwcrypto_digest_copy    copy,
    Iphwcrypto_digest_cleanup cleanup,
    int                       last
    );
```

Description

This routine adds a digest method to an engine.

Parameters:

id

The id of the engine.

type

The type of digest.

ctx_size

The number of bytes to be allocated for the context.

init

A pointer to the digest context initialization function.

update

A pointer to the digest update function.

final

A pointer to the digest final function.

copy

A pointer to the digest copy function.

cleanup

A pointer to the digest cleanup function.

last

A flag indicating the last digest to be added. 1 means that this is the last in the collection of digests to be added to the engine. 0 means that more will be added.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_NO_MEMORY** = out of memory, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_digests( ) - register added digest methods

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_digests
    (
    const char *id
    );
```

Description

This routine registers digest methods that have been added to an engine, and sets the engine as the default engine for digest operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_cbc_cipher( ) - add a CBC cipher method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_cbc_cipher
    (
    const char                *id,
    int                       type,
    Iphwcrypto_cbc_cipher_calc c,
    int                       last
    );
```

Description

This routine adds a CBC cipher method to an engine.

Parameters:

id

The id of the engine.

type

The type of cipher.

c

A pointer to the CBC cipher function.

last

A flag indicating the last cipher to be added. 1 means that this is the last in the collection of ciphers to be added to the engine. 0 means that more will be added.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_NO_MEMORY** = out of memory, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_ecb_cipher( ) - add an ECB cipher method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_ecb_cipher
    (
    const char                *id,
    int                       type,
    Iphwcrypto_ecb_cipher_calc c,
    int                       last
    );
```

Description

This routine adds an ECB cipher method to an engine.

Parameters:

id

The id of the engine.

type

The type of cipher.

c

A pointer to the ECB cipher function

last

A flag indicating the last cipher to be added. 1 means that this is the last in the collection of ciphers to be added to the engine. 0 means that more will be added.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_NO_MEMORY** = out of memory, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_ciphers( ) - register added cipher methods

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_ciphers
    (
    const char *id
    );
```

Description

This routine registers cipher methods that have been added to an engine and sets the engine as the default engine for cipher operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_random( ) - add random number generation method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_random
    (
    const char            *id,
    Iphwcrypto_random_gen r
    );
```

Description

This routine adds a random number generation method to an engine.

Parameters:

id

The id of the engine.

r

A pointer to the random number generation function.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_random( ) - register added random number generator

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_random
    (
    const char *id
    );
```

Description

This routine registers a random number generator that has been added to an engine and sets the engine as the default engine for random number generation.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_dh( ) - add Diffie-Hellman method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_dh
    (
    const char         *id,
    Iphwcrypto_mod_exp modexp
    );
```

Description

This routine adds a Diffie-Hellman method to an engine.

Parameters:

id

The id of the engine.

modexp

A pointer the modular exponentiation function.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_dh( ) - register added Diffie-Hellman method

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_dh
    (
    const char *id
    );
```

Description

This routine registers a Diffie-Hellmand method that has been added to an engine and sets the engine as the default engine for DH operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_rsa( ) - add RSA method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_rsa
    (
    const char         *id,
    Iphwcrypto_mod_exp modexp
    );
```

Description

This routine adds an RSA method to an engine.

Parameters:

id

The id of the engine.

modexp

A pointer the modular exponentiation function.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_rsa( ) - register added RSA method

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_rsa
    (
    const char *id
    );
```

Description

This routine registers an RSA method that has been added to an engine and sets the engine as the default engine for RSA operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_add_dsa( ) - add DSA method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_add_dsa
    (
    const char         *id,
    Iphwcrypto_mod_exp modexp
    );
```

Description

This routine adds a DSA method to an engine.

Parameters:

id

The id of the engine.

modexp

A pointer the modular exponentiation function.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_engine_register_dsa( ) - register added DSA method

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
int iphwcrypto_engine_register_dsa
    (
    const char *id
    );
```

Description

This routine registers a DSA method that has been added to an engine and sets the engine as the default engine for DSA operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_ipsec_engine_add_esp( ) - add IPsec ESP method to an engine

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
IP_PUBLIC Ip_err iphwcrypto_ipsec_engine_add_esp
    (
    const char           *id,
    int                  type,
    Iphwcrypto_ipsec_esp esp_in,
    Iphwcrypto_ipsec_esp esp_out
    );
```

Description

This routine adds an IPsec ESP method to an engine. Either the ESP input or the ESP output function pointers may be null.

Parameters:

id

The id of the engine.

type

The type of ESP.

esp_in

A pointer to the ESP input function.

esp_out

A pointer to the ESP output function.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_INVALID_ARG** = invalid argument, **IPCOM_ERR_FAILED** = other error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_ipsec_engine_register_esp( ) - register added IPsec ESP method

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
IP_PUBLIC Ip_err iphwcrypto_ipsec_engine_register_esp
    (
    const char *id
    );
```

Description

This routine registers an IPsec ESP method that has been added to an engine and sets the engine as the default engine for IPsec ESP operations.

Parameters:

id

The id of the engine.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_ipsec_esp_re_input( ) - re-input IPsec ESP packet

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
IP_PUBLIC Ip_err iphwcrypto_ipsec_esp_re_input
    (
    Iphwcrypto_esp_cbc_sa *sa,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt,
    UINT8                 *digest
    );
```

Description

This routine re-inputs an IPsec ESP packet after an asynchronous operation.

Parameters:

sa

A pointer to the SA (tunnel descriptor block).

param

A pointer to the IPsec parameters.

pkt

A pointer to the packet structure.

digest

A pointer the calculated digest.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_ipsec_esp_re_output( ) - re-output IPsec ESP packet

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
IP_PUBLIC Ip_err iphwcrypto_ipsec_esp_re_output
    (
    Iphwcrypto_esp_cbc_sa *sa,
    Ipipsec_param         *param,
    Ipcom_pkt             *pkt,
    UINT8                 *digest
    );
```

Description

This routine re-outputs an IPsec ESP packet after an asynchronous operation.

Parameters:

sa

A pointer to the SA (tunnel descriptor block).

param

A pointer to the IPsec parameters.

pkt

A pointer to the packet structure.

digest

A pointer the calculated digest.

Returns

IPCOM_SUCCESS = ok, **IPCOM_ERR_FAILED** = error

Errno

N/A

See also

iphwcrypto

Name

iphwcrypto_ipsec_esp_pkt_free( ) - return previously kept packet to the system

Layer

CRYPTOMISC_IPHWCRYPTO

Synopsis

```
IP_PUBLIC void iphwcrypto_ipsec_esp_pkt_free
    (
    Ipcom_pkt *pkt
    );
```

Description

This routine returns a previously kept packet to the system. It must be called if an asynchronous ESP operation fails or if the call to **iphwcrypto_ipsec_esp_re_output( )** or **iphwcrypto_ipsec_esp_re_input( )** fails.

Parameters:

pkt

A pointer to the packet structure.

Returns

Nothing.

Errno

N/A

See also

iphwcrypto

4. IPSCEP

Name

ipscep - Public API of Wind River IKE

Layer

SECURITY_MISC_SCEP

Routines

No user-callable routines

Description

IPSCEP_SUCCESS, **IPSCEP_INVALID_ARG**, **IPSCEP_ERR_FAILED**, **IPSCEP_ERR_TIMEOUT**, **IPSCEP_ERR_NO_MEMORY**, **IPSCEP_ERR_SERVER_UNAVAILABLE**, **IPSCEP_ERR_WRONG_URL**, **IPSCEP_ERR_WRONG_DN**, **IPSCEP_ERR_WRONG_KEY**, **IPSCEP_ERR_HTTP_SERVER_ERROR**, **IPSCEP_ERR_REQ_FAILURE_PENDING**, **IPSCEP_ERR_REQ_FAILURE_BAD_ALGH**, **IPSCEP_ERR_REQ_FAILURE_BAD_MSG_CHECK**, **IPSCEP_ERR_REQ_FAILURE_BAD_REQUEST**, **IPSCEP_ERR_REQ_FAILURE_BAD_TIME**, **IPSCEP_ERR_REQ_FAILURE_BAD_CERT_ID**, } Ipscep_err;

Include files

none

5. LOGINFUNCBIND

Name

loginFuncBind - **loginLib** function binding library

Layer

USER_MANAGEMENT

Routines

No user-callable routines

Description

This module provides **loginLib** facility function pointers.

Include files

private/loginLibP.h

6. LOGINLIB

Name

loginLib - user login/password subroutine library

Layer

USER_MANAGEMENT

Routines

loginUserVerify( ) - verify a user name and password in the user database **loginPrompt( )** - display a login prompt and validate a user entry **loginStringSet( )** - change the login string **loginAuthBlockedMsgSet( )** - change the blocked authentication message

Description

This library provides a login/password facility for the VxWorks shell (network and console access). When installed, it requires a user name and password match to gain access to the VxWorks shell from rlogin, telnet or the console. Therefore VxWorks can be used in secure environments where access must be restricted.

Routines are provided to prompt for the user name and password, and verify the response by looking up the name/password pair in a user database. This database contains the list of users that will be allowed to log in to the VxWorks shell locally or remotely.

If the user cannot be found in the local user database, the RADIUS protocol can be used to check the user's credentials near a RADIUS server. See the Network stack documentation regarding how to set up the IPCOM RADIUS protocol support.

Caution

Please note that the **SYSFLG_NO_SECURITY** boot flag, when set at the boot prompt, can disable shell security entirely within the system, even if configured at build time. This applies to all shell connections, be they remote or local. As this can be seen as a potential security hole, such boot flag support is enabled only if the **SYSFLG_NO_SECURITY_ENABLE** project parameter is set to **TRUE**. The default is to set it to **FALSE**, i.e **SYSFLG_NO_SECURITY** has no effect.

Configuration

The login security feature is initialized by the root task, **usrRoot( )**, in **usrConfig.c**, by means of the **INCLUDE_SECURITY** component. In addition, the **INCLUDE_SHELL_SECURITY** component allows to separately turn on security for the console shell. For a proper security setting configure VxWorks with both **INCLUDE_SECURITY** and **INCLUDE_SHELL_SECURITY**.

Adding additional users to the system

See the manual of the VxWorks User Management feature.

Logging in

When the login security facility is installed, every connection attempt, be it remote (login, telnet, SSH or FTP) or local (console shell) will make the VxWorks shell first prompt for a user name and password.

```
% rlogin target

VxWorks login: fred
```

```
    Password: flintstone

    ->
```

The delay in prompting between unsuccessful logins is increased linearly with the number of attempts, in order to slow down password-guessing programs.

When the user database does not exist or is empty, the first user attempting to log in will be prompted for creating the initial user in the system:

```
* Creation of initial user **

Initial user's login: fred
Password: <typed-in password is not echoed>
Enter password again: <typed-in password is not echoed>
```

All subsequent users attempting to log in will directly be prompted for their login name and password. The user database must therefore have been populated with them by the initial user prior to any other user logging-in.

Please note that starting with VxWorks 6.9, the shell no more sets the remote user identity after a successful login (it used to call **remCurIdSet( )** with the credentials used for login). If the remote user identity needs to be changed (possibly interactively), the **iam( )** shell routine can be used to that end.

Include files

loginLib.h

See also

shellLib, VxWorks User Management, VxWorks and Network Stack programmer's guides

Name

loginUserVerify( ) - verify a user name and password in the user database

Layer

USER_MANAGEMENT

Synopsis

```
STATUS loginUserVerify2
    (
    char *     name,     /* name of user */
    char *     passwd,   /* password of user */
    LOGIN_CTX* ctx       /* context with additional info */
    )
```

Description

This routine verifies a user entry in the user database. If configured the VxWorks User Management feature will be used. In case the user is not known locally, the RADIUS server will be used, if configured.

Returns

OK, or **ERROR** if the user name or password is not found.

Errno

Possible errnos set by this routine include:

EINVAL

An invalid argument is passed to the routine.

S_loginLib_UNKNOWN_USER

Unknown user name name.

S_loginLib_INVALID_PASSWORD

Invalid password passwd for name

See also

loginLib, **loginUserAdd( )**

Name

loginPrompt( ) - display a login prompt and validate a user entry

Layer

USER_MANAGEMENT

Synopsis

```
STATUS loginPrompt
    (
    char * userName   /* user name, ask if NULL or not provided */
    )
```

Description

This routine displays a login prompt and validates a user entry. If both user name and password match with an entry in the login table, the user is then given access to the VxWorks system. Otherwise, it prompts the user again.

All control characters are disabled during authentication except CTRL-D, which will terminate the remote login session.

Returns

OK if the name and password are valid, or **ERROR** if there is an **EOF** or the routine times out.

Errno

N/A

See also

loginLib, **iam( )**, **usrLib**

Name

loginStringSet( ) - change the login string

Layer

USER_MANAGEMENT

Synopsis

```
void loginStringSet
    (
    char * newString   /* string to become new login prompt */
    )
```

Description

This routine changes the login prompt string to newString. The maximum string length is **MAX_LOGIN_NAME_LEN** characters.

Returns

N/A

Errno

N/A

See also

loginLib

Name

loginAuthBlockedMsgSet( ) - change the blocked authentication message

Layer

USER_MANAGEMENT

Synopsis

```
void loginAuthBlockedMsgSet
    (
    char * newMsg   /* string to become new blocked auth message */
    )
```

Description

This routine changes the blocked authentication message to newMsg. The maximum message length is BUFSIZ characters, including the terminating EOS character.

Returns

N/A

Errno

N/A

See also

loginLib

7. LOGINPOLICYLIB

Name

loginPolicyLib - Login Policy Lib

Layer

USER_MANAGEMENT_POLICY

Routines

loginPolicyEnforcerAdd( ) - Add a login-policy-enforcer function callback **loginPolicyEnforcerDelete( )** - Remove a login-policy-enforcer function callback

Description

This module provides a framework to register login policy enforcers. The policy enforcers are callbacks that are invoked every time a login or logout request is received. A custom policy enforcer must implement this interface:

```
typedef STATUS (*LOGIN_POLICY_ENFORCER_CB)(const char* user, const char* pw, unsigned int even
t, const LOGIN_CTX* ctx);
```

The user and pw arguments hold the username and password provided in the login request. Event can take the following values

VALID_USER_PW

the username and pw are valid

INVALID_USER

the username is not defined in the user database

INVALID_PW

the username is defined in the user database, but the password is incorrect

LOGOUT

the user has logged out

If a policy enforcers returns **ERROR**, the login request will be denied.

A policy enforcer typically use some of the following APIs to gather information in order to determine if the login request is complying with policies:

pwValidGet(const **LOGIN_CTX*** ctx)

userDefinedGet(const **LOGIN_CTX*** ctx)

loginCtxInterfaceGet(**LOGIN_CTX*** ctx)

loginCtxFdGet(**LOGIN_CTX*** ctx)

In addition other APIs can be used when determining if a login request complies with a policy, e.g. a function that returns the time.

Include files

[**userMgntPolicy.h**]

Name

loginPolicyEnforcerAdd( ) - Add a login-policy-enforcer function callback

Layer

USER_MANAGEMENT_POLICY

Synopsis

```
STATUS loginPolicyEnforcerAdd
    (
    const LOGIN_POLICY_ENFORCER_CB policyEnforcerCb   /* Policy enforcer callback */
    )
```

Description

This function adds a login-policy-enforcer function callback to the list of enforcer callbacks that are called when someone attempts to log in.

Returns

OK on success, **ERROR** otherwise

Errno

ENOMEM

Not enough memory available to add the callback

EEXIST

The callback is already registered

See also

loginPolicyLib

Name

loginPolicyEnforcerDelete( ) - Remove a login-policy-enforcer function callback

Layer

USER_MANAGEMENT_POLICY

Synopsis

```
STATUS loginPolicyEnforcerDelete
    (
    const LOGIN_POLICY_ENFORCER_CB cb   /* Policy enforcer callback */
    )
```

Description

This function removes a login-policy-enforcer function callback, previously added by a call to loginPolicyEnforcerAdd

Returns

OK on success, **ERROR** otherwise

Errno

ENOENT

The callback is not in the list of registered callbacks

See also

loginPolicyLib

8. OPTEEAPI

Name

opteeAPI - OP-TEE Client API

Layer

OP_TEE

Routines

TEEC_InitializeContext( ) - Initialize TEE Client Context **TEEC_FinalizeContext( )** - Finalize TEE Client Context **TEEC_OpenSession( )** - Open a TEE Client API Session **TEEC_CloseSession( )** - Close a TEE Client API Session **TEEC_InvokeCommand( )** - Invoke TEE Client API command **TEEC_RequestCancellation( )** - Cancel a previously issued TEE Client API command **TEEC_RegisterSharedMemory( )** - Register TEE Client API Shared Memory **TEEC_AllocateSharedMemory( )** - Allocate TEE Client API Shared Memory **TEEC_ReleaseSharedMemory( )** - Release TEE Client API Shared Memory

Description

These file provides the implementation OP-TEE Client API.

Include files

none

Name

TEEC_InitializeContext( ) - Initialize TEE Client Context

Layer

OP_TEE

Synopsis

```
TEEC_Result TEEC_InitializeContext
    (
    const char *   name,
    TEEC_Context * context
    )
```

Description

This routine implements the **TEEC_InitializeContext( )** API defined in TEE Client API Specification V1.0.

Returns

listed in the TEE Client API Sepcification V1.0

Errno

N/A

See also

opteeAPI

Name

TEEC_FinalizeContext( ) - Finalize TEE Client Context

Layer

OP_TEE

Synopsis

```
void TEEC_FinalizeContext
    (
    TEEC_Context * context
    )
```

Description

This routine implements the **TEEC_FinalizeContext( )** API defined in TEE Client API Specification V1.0.

Returns

N/A

Errno

N/A

See also

opteeAPI

Name

TEEC_OpenSession( ) - Open a TEE Client API Session

Layer

OP_TEE

Synopsis

```
TEEC_Result TEEC_OpenSession
    (
    TEEC_Context *    context,
```

```
    TEEC_Session *    session,
    const TEEC_UUID * destination,
    uint32_t          connectionMethod,
    const void *      connectionData,
    TEEC_Operation *  operation,
    uint32_t *        retOrigin
    )
```

Description

This routine implements the **TEEC_OpenSession( )** API defined in TEE Client API Specification V1.0.

Returns

listed in the TEE Client API Sepcification V1.0

Errno

N/A

See also

opteeAPI

Name

TEEC_CloseSession( ) - Close a TEE Client API Session

Layer

OP_TEE

Synopsis

```
void TEEC_CloseSession
    (
    TEEC_Session * session
    )
```

Description

This routine implements the **TEEC_CloseSession( )** API defined in TEE Client API Specification V1.0.

Returns

N/A

Errno

N/A

See also

opteeAPI

Name

TEEC_InvokeCommand( ) - Invoke TEE Client API command

Layer

OP_TEE

Synopsis

```
TEEC_Result TEEC_InvokeCommand
    (
    TEEC_Session *   session,
    uint32_t         cmdId,
    TEEC_Operation * operation,
    uint32_t *       retOrigin
    )
```

Description

This routine implements the **TEEC_InvokeCommand( )** API defined in TEE Client API Specification V1.0.

Returns

listed in the TEE Client API Sepcification V1.0

Errno

N/A

See also

opteeAPI

Name

TEEC_RequestCancellation( ) - Cancel a previously issued TEE Client API command

Layer

OP_TEE

Synopsis

```
void TEEC_RequestCancellation
    (
    TEEC_Operation * operation
    )
```

Description

This routine implements the **TEEC_RequestCancellation( )** API defined in TEE Client API Specification V1.0.

Returns

N/A

Errno

N/A

See also

opteeAPI

Name

TEEC_RegisterSharedMemory( ) - Register TEE Client API Shared Memory

Layer

OP_TEE

Synopsis

```
TEEC_Result TEEC_RegisterSharedMemory
    (
    TEEC_Context *      context,
    TEEC_SharedMemory * shm
    )
```

Description

This routine implements the **TEEC_RegisterSharedMemory( )** API defined in TEE Client API Specification V1.0.

Returns

listed in the TEE Client API Sepcification V1.0

Errno

N/A

See also

opteeAPI

Name

TEEC_AllocateSharedMemory( ) - Allocate TEE Client API Shared Memory

Layer

OP_TEE

Synopsis

```
TEEC_Result TEEC_AllocateSharedMemory
    (
    TEEC_Context *      context,
    TEEC_SharedMemory * shm
    )
```

Description

This routine implements the **TEEC_AllocateSharedMemory( )** API defined in TEE Client API Specification V1.0.

Returns

listed in the TEE Client API Sepcification V1.0

Errno

N/A

See also

opteeAPI

Name

TEEC_ReleaseSharedMemory( ) - Release TEE Client API Shared Memory

Layer

OP_TEE

Synopsis

```
void TEEC_ReleaseSharedMemory
    (
    TEEC_SharedMemory * shm
    )
```

Description

This routine implements the **TEEC_ReleaseSharedMemory( )** API defined in TEE Client API Specification V1.0.

Returns

N/A

Errno

N/A

See also

opteeAPI

9. OPTEECORE

Name

opteeCore - OP-TEE Client API

Layer

OP_TEE

Routines

No user-callable routines

Description

These routines are the OP-TEE Client API core logic in kernel space.

Include files

none

10. OPTEEIODEVICE

Name

opteeIoDevice - OP-TEE Client API

Layer

OP_TEE

Routines

No user-callable routines

Description

These file provides the IO Device wrapper layer for OP-TEE Client API.

Include files

none

11. OPTEERPC

Name

opteeRpc - OP-TEE Client API RPC services

Layer

OP_TEE

Routines

No user-callable routines

Description

These routines implement the OP-TEE Remote Procedure Call services.

Include files

none

12. PRIVILEGESLIB

Name

privilegesLib - VxWorks privileges library

Layer

USER_MANAGEMENT_USER_PRIVILEGES

Routines

privilegesCheck( ) - check user's privileges against a given hierarchy of objects

Description

This library implements the privilege and permission support. Privileges are described via entries in the Privilege Manifest file.

This file describes the privileges associated with the listed set of objects. The privilege descriptions inform the system as to who may do what with which object.

The overall syntax for a privilege description entry is as follows:

```
<object hierarchy>:<user list>:<group list>
```

Object hierarchy

An object hierarchy is a dot-separated list of objects:

```
<object_1>.<object_2>.<object_3>...
```

By convention the first object and last object in the hierarchy have a special meaning.

The first object in the list is known as the root object and represents a "category". A category is a way to avoid entry lookup collisions and are generally associated to a runtime facility. Some category names are reserved for VxWorks (e.g. "shell") and should not be used for privilege description entries not intended to apply to such a facility.

The last object in the hierarchy is known as an "operation", i.e. a function meaningful to that object hierarchy.

The objects listed in a hierarchy are all logically connected and are presented in subordination order: the last object applies to the previous object which applies to the previous object and so on, till the root object.

Several object hierarchies may exist for a given category, and different operations may apply to each object hierarchy.

Except for the category, any other object in a hierarchy may be represented by the wildcard character '*****'. The wildcard character represents all the objects available for the level of the hierarchy where the wildcard character is used. For instance using the wildcard character for the operation applying to a hierarchy means that the entry applies for all the operations applicable to the hierarchy.

Note: the '*****' character used in this situation is not to be understood as similar to the '*****' metacharacter used in POSIX regular expressions. In an object hierarchy the wildcard character stands alone.

An object hierarchy may be truncated from last to second object, i.e. from the operation to the category, non included for obvious reasons. A truncated object hierarchy is interpreted as if the wildcard character had been used for all the non-described objects in that hierarchy.

Therefore the following entries are strictly equivalent and all mean that the entry applies to all the operations supported by the entire object hierarchy in that category:

```
<category>
<category>.*
<category>.*.*
<category>.*.*.*
```

For example, in the shell category, the second object is a shell command, while the other objects in the hierarchy represent items meaningful to that command, with the last object describing an operation that applies to the shell command's object hierarchy, usually execute.

An object hierarchy may also be truncated of all or several objects between the last object, the operation, and the first object, the category. A truncated object hierarchy is then interpreted as if the wildcard character had been used for all the non-described objects in that hierarchy.

Therefore the following entries are strictly equivalent and all mean that the entry applies to all the operations supported by the entire object hierarchy in that category:

```
<category>.execute
<category>.*.execute
<category>.*.*.execute
```

For example, in the shell category, the second object is a shell command, while the other objects in the hierarchy represent items meaningful to that command, with the execute operation.

The entry shell.module.info.execute indicates that the execute operation applies to the shell's **module info** command. The entry shell.module.*.execute indicates that the execute operation applies to all the functionality of the **module** command.

Hierarchy entries with more described objects than the hierarchy for which privileges are checked won't be considered in the verification process. For example, the shell's **module info** command will match the following entries. (Note the parsing described in the PRIVILEGE VERIFICATION won't actually go to the entry, shell.module.info.*.execute, if the shell.module.info.execute is listed prior to the entry in the privilege manifest since the shell.module.info.execute will be found as an exact match and the parse will immediately stop.)

```
shell
shell.*
shell.*.*
shell.*.*.*
shell.*.*.*.*
shell.execute
shell.*.execute
shell.*.*.execute
shell.*.*.*.execute
shell.module
shell.module.*
shell.module.*.*
shell.module.*.*.*
shell.module.info
shell.module.info.*
shell.module.info.*.*
shell.module.execute
shell.module.*.execute
shell.module.*.*.execute
shell.module.info.execute
shell.module.info.*.execute
```

However, none of the following entries will be matched.

```
shell.*.*.*.*.*
shell.*.*.*.*.execute
shell.module.*.*.*.*
shell.module.*.*.*.execute
shell.module.info.*.*.*
shell.module.info.*.*.execute
```

User and group lists

Who may do the indicated operations on the listed object hierarchies is described by lists associated with each object hierarchy (i.e. on the same line). Each list is separated from each other and from the object hierarchy by the colon character '**:**'. Lists are always in this precise order: first the user list, then the group list.

A list is a comma-separated set of names or numerical IDs. In some conditions a list may be empty, in which case its content is not evaluated.

Note that a root user (super user) created with the user ID of '**1**' and the group ID of '**1**' will always have permissions on all object hierarchies. The root user (super user) may have any user name, as long as that user name is associated with a '**1**' for both user ID and group ID.

User list

The user list is a comma-separated set of user names or numerical user IDs:

```
<user_1_name_or_ID>,<user_2_name_or_ID>,<user_3_name_or_ID>...
```

Group list

The group list is a comma-separated set of numerical group IDs (group names are not supported):

```
<group_1_ID>,<group_2_ID>,<group_3_ID>...
```

Those names or IDs correspond to user's names or IDs, and group's IDs as registered in the VxWorks user database.

The group list is separated from the user list by the colon character '**:**'.

```
<user list>:<group list>
```

The user list may be empty only if it is followed by a group list. This allows all users in the group list to perform an operation on the object hierarchy:

```
<object hierarchy>::<group list>
```

The group list may be empty if the user list is non-empty, with user names specified. Only users in the user list are allowed to perform an operation on an object hierarchy.

```
<object hierarchy>:<user list>
```

The group list may also be empty if no user is allowed to perform an operation on an object hierarchy and no group list is provided for that hierarchy. The user list must only hold the numerical **0** value:

```
<object hierarchy>:0
```

On the other hand, if all users are allowed to perform an operation on an object hierarchy, the wildcard character '*' may be used in that hierarchy's user list to represent all users.

```
<object hierarchy>:*
```

Similar to the user list, the wildcard character '*' may be used to represent all groups in the group list. Note however that this would prevent anything provided in the user list to be effective for the verification of privileges.

```
<object hierarchy>:<user list>:*
```

The following are some examples of using the user list and group list:

```
# Only users with user name "user_1", or with user ID '2', or with group ID
# '100' or with group ID '101' are allowed to perform this shell command.
shell.module.info:user_1,2:100,101

# Only users with group ID '100' are allowed to perform this shell command.
shell.module.info::100

# Only users with user name "user_1", or with user ID '2', are allowed to
# perform this shell command.
shell.module.info:user_1,2

# No user is allowed to perform this shell command.
shell.module.info:0

# All users are allowed to perform this shell command.
shell.module.info:*

# All users are allowed to perform this shell command.
shell.module.info::*
```

Privilege verification

Each time the verification of a user's privileges for a given hierarchy is requested, via the **privilegesCheck( )** API, the object hierarchies listed in the privilege manifest are parsed. A user is deemed to have appropriate privileges when the following condition 1) is true and either condition 2) or condition 3) is true.

1) a match is found between the hierarchy for which user's privileges are checked and a hierarchy listed in the privilege manifest, and

2) either the user is listed by name or ID in that hierarchy's user list or the wildcard character '*' is found, or

3) the user's group ID is listed in that hierarchy's group list or the wildcard character '*' is found.

Note that the **privilegesCheck( )** API always grants permission when its caller has the root user's identity.

When an exact match (i.e. identical strings) is found between the hierarchy for which user's privileges are checked and a hierarchy listed in the privilege manifest, the parsing immediately stops, whether the user has appropriate privileges for that hierarchy or not. This "stop on exact match" behavior implies that only one instance of a given object hierarchy must be

provided in the manifest otherwise there is a possibility for the privilege verification process to lead to denial of permission for a user not listed in the user list of the first appearing instance of an object hierarchy although that user is listed against another instance of that same object hierarchy.

The command that takes an argument may require an additional object as '*****' to be found as an exact match. If the execute is specified at the last object as an operation and the '*****' is not specified as the object describing an argument, the '**execute**' is interpreted as if the wildcard character had been specified before the '**execute**', like '***.execute**'. Therefore for instance, either the entry shell.module.info.execute or shell.module.info.*.execute is found as an exact match for the **module info** shell command.

Unless the caller has the root user's identity or an exact match is found, the **privilegesCheck( )** API will parse all the entries from the top to the end of the privilege manifest to find the most specific entry to verify the privilege. The **privilegesCheck( )** API judges the best fit by finding continuous matches from the first object and counting the objects till the last object in the hierarchy it is evaluating. If there are several entries evaluated as at the same highest matching level, the first entry listed in the privilege manifest will be used. For instance, for the shell command **module info**, the following entries can be categorized as one to three level(s) hierarchy matched entries. The first of the three level hierarchy matched entries, that is shell.module.info, will be used for the privilege verification in the following example.

```
# One level hierarchy is matched.
shell
shell.*
shell.*.*
shell.*.*.*
shell.*.*.*.*
shell.execute
shell.*.execute
shell.*.*.execute
shell.*.*.*.execute
shell.*.info
shell.*.info.*
shell.*.info.*.*
shell.*.info.execute
shell.*.info.*.execute

# Two levels hierarchy are matched.
shell.module
shell.module.*
shell.module.*.*
shell.module.*.*.*
shell.module.execute
shell.module.*.execute
shell.module.*.*.execute

# Three levels hierarchy are matched.
shell.module.info
shell.module.info.*
shell.module.info.*.*
```

Because wildcards characters are generally used in the manifest's privilege description entries, and can be used as well in the object hierarchy passed to the **privilegesCheck( )** API, the more restrictive object hierarchies should be listed more specifically (i.e. without using wildcards) so as not to grant privileges inappropriately. For instance, the object hierarchies related to the **set config** shell command must be listed as follows if the ability to change the default shell interpreter must be restricted to an "admin" user while all other users may use the other features supported by the **set config** shell command:

```
shell.set.config.INTERPRETER.C:admin
shell.set.config.*.*:*
```

Similarly, in order to allow only the "admin" user to delete files as well as create and delete directories via the FTP and SFTP services, while allowing all other users to do all other operations, the following entries would be needed:

```
ftp.MKD.*.execute:admin
ftp.RMD.*.execute:admin
ftp.DELE.*.execute:admin
ftp.*.*.*:*
sftp.MKDIR.*.execute:admin
sftp.RMDIR.*.execute:admin
sftp.REMOVE.*.execute:admin
sftp.*.*.*:*
```

Object hierarchies not listed in the privilege manifest are considered disallowed for all users.

Syntax

Syntax of a privilege description entry:

```
<category>[.<object>[.<...>].<operation>]:<user>[,<...>][:<group>[,<...>]]
```

Limits

The maximum length of a privilege description entry is 255 characters.

The maximum number of objects in a hierarchy is 10.

The maximum length of a hierarchy's object name is 40 characters.

The maximum length of a user name is **MAX_LOGIN_NAME_LEN** (80) characters.

Configuration

For convenience, a privilege manifest template is provided in the VxWorks Image Project (VIP) when the **INCLUDE_USER_PRIVILEGES** component is included. This template is located in the privilege_manifest subdirectory which appears after the VIP has been built once. Once the template has been adjusted to the user's requirements it can be copied in the location indicated by the **PRIVILEGE_MANIFEST_PATH** parameter. Note that, by default, the privilege manifest is expected to be found on ROMFS (/romfs/prvlgManifest.txt) so it can be convenient to simply rename the privilege_manifest subdirectory into romfs. Once the **INCLUDE_ROMFS** component is included and the VIP is built, the content of that romfs directory will automatically be part of the ROMFS payload.

Do note that a copy of the adjusted privilege manifest must be done in a safe location prior to deleting the VIP as this will also delete the manifest.

Note also that when **USER_PRIVILEGES** is selected in a VxWorks Source Build (VSB) project, the default shell interpreter is automatically changed to being the command interpreter regardless of whether or not the **INCLUDE_USER_PRIVILEGES** component is included in the VIP. Additionally, when the **INCLUDE_USER_PRIVILEGES** component is actually included in a VIP, the trap-to-monitor character (CTRL-X by default) is disabled so the system can be rebooted only via the **reboot** command.

Include files

none

Name

privilegesCheck( ) - check user's privileges against a given hierarchy of objects

Layer

USER_MANAGEMENT_USER_PRIVILEGES

Synopsis

```
STATUS privilegesCheck
    (
    char * objHier,    /* full or truncated object hierarchy */
    char * userName,   /* caller's user name. May be set to NULL if unused */
    uid_t  userId,     /* caller's UID. May be set to zero if unused */
    BOOL * pResult     /* TRUE if the caller has appropriate privileges */
    )
```

Description

This routine checks whether the user has appropriate privileges to do operations on the objHier object hierarchy. The requested operation is specified by the object hierarchy passed to this routine. The caller's user name is specified either by a login name or a user ID. If both are provided they must match each other or privilege verification will fail. If none are provided or only the user name is provided, it is assumed that it is the caller's user identity that must be used to check privileges.

The object hierarchy may be full or truncated (i.e. not all the elements listed). It also may hold wildcard characters for any object but the root object (<category>), however those characters are matched against wildcard characters in the privilege manifest's entries, so permission will not be granted if the manifest's entries are more specific than the object hierarchy for which appropriate privileges are checked.

Whether the user may do the sought operation depends on either of the following condition pairs 1) and 2) or 1) and 3) being successful:

1) a match can be found between the objHier object hierarchy and at least one of the privilege description entries of the privilege manifest.

2) the user's name or ID is found in the matching privilege description entry (a wildcard character will positively match).

3) the user's group ID is found in the matching privilege description entry (a wildcard character will positively match).

Whether the caller has appropriate privileges for the requested object hierarchy is indicated via the pResult parameter, which must therefore point to a valid memory storage. This parameter holds **TRUE** if the caller has appropriate privileges, **FALSE** otherwise. Note that the pResult parameter is always set to **FALSE** when this routine returns **ERROR**. Check the error number to know the trouble's origin.

Note also that when the caller's identity is that of the root user, that caller is always deemed to have appropriate privileges.

Returns

OK if the privilege verification could be done, **ERROR** otherwise.

Errnos

S_privilegesLib_INVALID_PARAMETER

One or both of the objHier or pResult parameters are not valid (either **NULL** or, for objHier, holding an empty string.

S_privilegesLib_INVALID_USER_NAME

The userName parameter is **NULL** while the userId parameter is left to zero, or the user name is unknown in the user database.

S_privilegesLib_INVALID_USER_ID

The userId parameter holds a non-zero value which either does not correspond to the user name passed via the userName parameter or, if the userName parameter is left **NULL**, does not exist in the user database.

S_privilegesLib_MISSING_PRIVILEGE_MANIFEST

The privilege manifest could not be found where specified by the **PRIVILEGE_MANIFEST_PATH** CDL parameter.

See also

privilegesLib

13. PWPOLICYLIB

Name

pwPolicyLib - Password policy enforcer lib

Layer

USER_MANAGEMENT_POLICY

Routines

pwPolicyEnforcerAdd( ) - Add a password-policy enforcer function callback **pwPolicyEnforcerDelete( )** - Remove a password-policy enforcer callback

Description

The Password policy enforcer lib manages the registration and invokation of password-policy-enforcer callbacks.

Include files

none

Name

pwPolicyEnforcerAdd( ) - Add a password-policy enforcer function callback

Layer

USER_MANAGEMENT_POLICY

Synopsis

```
STATUS pwPolicyEnforcerAdd
    (
    const PW_CHANGE_POLICY_ENFORCER_CB policyEnforcerCb   /* policy enforcer callback */
    )
```

Description

This function adds a password-policy enforcer callback to the list of enforcer callbacks that are called when an account is created or a password is changed.

Returns

OK on success, **ERROR** otherwise

Errno

ENOMEM

Not enough memory available to add the callback

EEXIST

The callback is already registered

See also

pwPolicyLib

Name

pwPolicyEnforcerDelete( ) - Remove a password-policy enforcer callback

Layer

USER_MANAGEMENT_POLICY

Synopsis

```
STATUS pwPolicyEnforcerDelete
    (
    const PW_CHANGE_POLICY_ENFORCER_CB cb   /* policy enforcer callback */
    )
```

Description

This function removes a password-policy enforcer callback, previously added by a call to pwPolicyEnforcerAdd

Returns

OK on success, **ERROR** otherwise

Errno

ENOENT

The callback is not in the list of registered callbacks

See also

pwPolicyLib

14. SECCIPHER

Name

secCipher - VxWorks Symmetrical Encryption Library

Layer

SECURITY_MISC_SEC_CRYPTO

Routines

secCipherInit( ) - Initialize a cipher context **secCipherUpdate( )** - Encrypt or decrypt data **secCipherFinal( )** - Finalize Encryption/Decryption **secCipherCtrl( )** - Perform ioctl-like operations on a **SEC_CIPHER_CTX** **secCipherCleanup( )** - Cleanup an initialized context **secCipherCopy( )** - Copies a cipher context

Description

This module is the VxWorks symmetrical encryption library. The library separates the application using an encryption algorithm from the provider. The application specifies what encryption algorithm to use by means of a template. The template is used to initialize a cipher context. The API is designed in such a way that once the cipher context is initialized, the application does not need keep track of the encryption algorithm or mode used. That is all encapsulated by the library.

Below is a code example encrypting two clear text buffers using AES CBC with a 128 bit key:

```
SEC_CIPHER_CTX ctx;
unsigned char encrData[1024];
int encrLen = 0;
int outLen;
int tmpLen = sizeof(encrData);

secCipherInit(&ctx, vxCipherAes128CbcTemplateGet(), key, iv, 1);
secCipherUpdate(&ctx, encrData, &tmpLen, clearTextData, inLen);
encrLen = tmpLen;
outLen = sizeof(encrData)-encrLen;
secCipherUpdate(&ctx, encrData+encrLen, &outlen, clearTextData2, inLen2);
encrLen += outLen;
outLen = sizeof(encrData)-encrLen;
secCipherFinal(&ctx, encrData+encrLen, &outLen);
encrLen += outLen;
secCipherCleanup(&ctx);

/* encrData now holds the encrypted data. encrLen holds the number of bytes written to encrDat
a */
```

Template functions

The following template functions exist:

```
const SEC_CIPHER_TEMPLATE* secCipherAes128CbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes128CtrTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes128EcbTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes192CbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes192CtrTemplateGet(void);
```

```
const SEC_CIPHER_TEMPLATE* secCipherAes256CbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes256CtrTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherAes256EcbTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherBlowfishCbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherCastCbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherDesCbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipher3DesCbcTemplateGet(void);
const SEC_CIPHER_TEMPLATE* secCipherRc4TemplateGet(void);
```

A proprietary implementation of a hash algorithms can be installed by calling set template function for that particular algorithm. The following set template functions exist:

```
void secCipherAes128CbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes128CtrTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes128EcbTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes192CbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes192CtrTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes256CbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes256CtrTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAes256EcbTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherAesCtrTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherBlowfishCbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherCastCbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherDesCbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipher3DesCbcTemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
void secCipherRc4TemplateSet(const SEC_CIPHER_TEMPLATE* ctx);
```

Include files

[**secCrypto.h**]

Name

secCipherInit( ) - Initialize a cipher context

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherInit
    (
    SEC_CIPHER_CTX*           ctx,
    const SEC_CIPHER_TEMPLATE* cipherTemplate,
    const unsigned char*      key,
    const unsigned char*      iv,
    const int                 encr
    )
```

Description

This function initializes a cipher context. After the context has been initialized it can be used to encrypt/decrypt data using the secCipherUpdate function. The context will by default be setup for PKCS padding. See secCipherCtrl for how to disable PKCS padding. Note that a context that has been successfully initialized must be freed by calling secCipherCleanup.

Arguments

ctx

pointer to the context that will be initialized

cipherTemplate

template controlling what encryption algorithm to use, e.g. the return value of **secCipherAes128CbcTemplateGet( )**.

key

pointer to the encryption key. It is the responsibility of the caller to ensure that the key is sufficiently long.

iv

initialization vector, required by some algorithms/modes. It is the responsibility of the caller to ensure that the iv is sufficiently long.

encr

a non-zero value means that the context will be used for encryption, otherwise it will be setup for decryption.

Returns

OK if the context was successfully initialized, **ERROR** if it failed.

Errno

N/A

See also

secCipher

Name

secCipherUpdate( ) - Encrypt or decrypt data

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherUpdate
    (
    SEC_CIPHER_CTX*      ctx,
    unsigned char*       out,
    int*                 outLen,
    const unsigned char* in,
```

```
    int                     inLen
    )
```

Description

This function encrypts or decrypts data. If data is to be encrypted or decrypted, and what encryption algorithm to use is specified in the initialization of ctx. see secCipherInit. Note that the number of bytes written to out can be both larger or smaller than inLen, when PKCS padding is enabled for block ciphers. At most, the output length can differ by one block from the input length. If a block cipher is used without padding, then the input length must be a multiple of the block size.

Arguments

ctx

pointer to the crypto context

out

pointer to buffer where the output will be stored

outLen

pointer to an integer where the output length will be stored

in

pointer to input data

inLen

number of input bytes

Returns

OK if the encryption/decryption was successful, **ERROR** otherwise

Errno

N/A

See also

secCipher

Name

secCipherFinal( ) - Finalize Encryption/Decryption

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherFinal
    (
```

```
    SEC_CIPHER_CTX* ctx,
    unsigned char*  out,
    int*            outLen
    )
```

Description

This function finalizes an encryption/decryption operation consisting of a sequence of secCipherInit, secCipherUpdate, secCipherUpdate ..., secCipherFinal. The main reason why this function is needed, is to handle PKCS padding.

Arguments

ctx

pointer to the crypto context

out

pointer to buffer where the output will be stored

outLen

pointer to an integer where the output length will be stored

Returns

OK on success, **ERROR** otherwise

Errno

N/A

See also

secCipher

Name

secCipherCtrl( ) - Perform ioctl-like operations on a **SEC_CIPHER_CTX**

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherCtrl
    (
    SEC_CIPHER_CTX* ctx,
    int             type,
    void*           arg,
    unsigned int*   len
    )
```

Description

This function is used to perform ioctl-like operations on a **SEC_CIPHER_CTX**. The following operations are defined:

SEC_CIPHER_CTRL_SET_PADDING

enable/disable PKCS padding. arg should of type int. 0 disables padding, a non-zero value enables padding.

SEC_CIPHER_CTRL_GET_PADDING

get PKCS padding setting. arg should be of type int*. The integer will be set to zero if padding is disabled, and one if padding is enabled.

SEC_CIPHER_CTRL_GET_IV

get a pointer to the initialization vector. arg should be of type unsigned char**

SEC_CIPHER_CTRL_SET_IV

set the initialization vector. arg should be of type unsigned char*

Returns

OK if the operation was successfully performed, **ERROR** otherwise

Errno

N/A

See also

secCipher

Name

secCipherCleanup( ) - Cleanup an initialized context

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherCleanup
    (
    SEC_CIPHER_CTX* ctx
    )
```

Description

This function cleans up an initialized context.

Returns

OK if the operation was successfully performed, **ERROR** otherwise

Errno

N/A

See also

secCipher

Name

secCipherCopy( ) - Copies a cipher context

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secCipherCopy
    (
    SEC_CIPHER_CTX* copy,
    SEC_CIPHER_CTX* org
    )
```

Description

This function copies a cipher context. The state, e.g. working IV, is included in the copy.

Returns

OK if the operation was successfully performed, **ERROR** otherwise

Errno

N/A

See also

secCipher

15. SECHASH

Name

secHash - VxWorks hash library

Layer

SECURITY_MISC_SEC_HASH

Routines

secHash( ) - Calculate a hash digest **secHashInit( )** - Initialize a hash context **secHashUpdate( )** - Add data to a hash calculation **secHashFinal( )** - Finalize a hash calculation **secHashCtrl( )** - Perform an ioctl-like operation **secHashCopy( )** - Copy a hash context **secHashCleanup( )** - Cleans up a hash context

Description

This file implements the VxWorks hash API. The API separates the application using a hash algorithm from the provider. The application specifies what hash algorithm to use by means of a template. The template is used to initialize a cipher context. The API is designed in such a way that once the hash context is initialized, the application does not need keep track of the hash algorithm used. That is all encapsulated by the API.

A typical usage of the hash API is shown below. It shows how to hash three buffers using SHA256.

```
SEC_HASH_CTX ctx;
unsigned char[32] digest;
unsigned int dgst_len = sizeof(digest);

secHashInit(&ctx, secHashSha256TemplateGet());
secHashUpdate(&ctx, data1, len1);
secHashUpdate(&ctx, data2, len2);
secHashUpdate(&ctx, data3, len3);
secHashFinal(&ctx, digest, &dgst_len);
secHashCleanup(&ctx);
```

Template functions

The following template functions exist:

```
const SEC_HASH_TEMPLATE* secHashMd5TemplateGet(void);
const SEC_HASH_TEMPLATE* secHashRmdTemplateGet(void);
const SEC_HASH_TEMPLATE* secHashSha1TemplateGet(void);
const SEC_HASH_TEMPLATE* secHashSha256TemplateGet(void);
const SEC_HASH_TEMPLATE* secHashSha384TemplateGet(void);
const SEC_HASH_TEMPLATE* secHashSha512TemplateGet(void);
```

A proprietary implementation of a hash algorithms can be installed by calling set template function for that particular algorithm. The following set template functions exist:

```
void secHashMd5TemplateSet(const SEC_HASH_TEMPLATE* ctx);
void secHashRmdTemplateSet(const SEC_HASH_TEMPLATE* ctx);
void secHashSha1TemplateSet(const SEC_HASH_TEMPLATE* ctx);
```

```
void secHashSha256TemplateSet(const SEC_HASH_TEMPLATE* ctx);
void secHashSha384TemplateSet(const SEC_HASH_TEMPLATE* ctx);
void secHashSha512TemplateSet(const SEC_HASH_TEMPLATE* ctx);
```

Include files

[**secHash.h**]

Name

secHash( ) - Calculate a hash digest

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHash
    (
    const void*              data,
    size_t                   len,
    unsigned char*           digest,
    unsigned int             *digestLen,
    const SEC_HASH_TEMPLATE* hash_template
    )
```

Description

This function can only be used if the data to hash is in one continuous buffer. secHashInit, secHashUpdate, secHashFinal and secHashCleanup must be used if the data to hash is stored in two or more buffers.

Arguments

data

pointer to the data that the will be included in the hash calculation

len

number of bytes to include in the hash calculation

digest

pointer to the buffer where the digest (or hash) data will be written

digestLen

length of the digest buffer

hash_template

controls the hash algorithm to use, e.g. use the return value of **secHashSha1TemplateGet( )**.

Returns

OK if the hash was successfully calculated, **ERROR** otherwise.

Errno

N/A

See also

secHash

Name

secHashInit( ) - Initialize a hash context

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHashInit
    (
    SEC_HASH_CTX*            ctx,
    const SEC_HASH_TEMPLATE* hash_template
    )
```

Description

This function initializes a hash context. The hash algorithm to use is specified by the hash_template argument. For instance, in order to use SHA1, the return value of **secHashSha1TemplateGet( )** should be used as template. Not that every successful call to secHashInit must be matched with a call to secHashCleanup.

Once the hash context has been setup, call secHashUpdate to add data to the hash calculation. Once all data has been added, call secHashFinal to get the resulting hash value. And finally call secHashCleanup in order to free any resources that has been allocated during the hash calculation.

Returns

OK if the initialization is successful. **ERROR** if the hash context could not be initialized.

Errno

N/A

See also

secHash

Name

secHashUpdate( ) - Add data to a hash calculation

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHashUpdate
    (
    SEC_HASH_CTX* ctx,
    const void*   data,
    size_t        count
    )
```

Description

This function adds data to a hash calculation. It can be called multiple times in order to add several memory segments to the calculation

Returns

OK is the data was successfully added, **ERROR** otherwise.

Errno

N/A

See also

secHash

Name

secHashFinal( ) - Finalize a hash calculation

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHashFinal
    (
    SEC_HASH_CTX*  ctx,
    unsigned char* digest,
    unsigned int*  size
    )
```

Description

Return the hash of the data added by calls to secHashUpdate.

Returns

OK if the hash value was successfully written to digest, **ERROR** otherwise.

Errno

N/A

See also

secHash

Name

secHashCtrl( ) - Perform an ioctl-like operation

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHashCtrl
    (
    struct SEC_HASH_CTX_ST* ctx,
    int                     type,
    void*                   arg,
    unsigned int*           len
    )
```

Description

This function performs an ioctl-like operation on the hash context. This is included to make the hash API future proof. There are currently no ctrl operations defined for hash contexts.

Returns

OK if the operation was successfully performed, **ERROR** otherwise.

Errno

N/A

See also

secHash

Name

secHashCopy( ) - Copy a hash context

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHashCopy
    (
    SEC_HASH_CTX* copy,
    SEC_HASH_CTX* org
    )
```

Description

This function makes a copy of a hash context. The hash context state is copied too. This means that if data is added to a hash context by calling secHashUpdate this data will be included in the hash calculation of the hash context copy too. Note that secHashCleanup must be called on the copy too.

Returns

OK if the context was successfully copied, **ERROR** otherwise.

Errno

N/A

See also

secHash

Name

secHashCleanup( ) - Cleans up a hash context

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
void secHashCleanup
    (
    SEC_HASH_CTX* ctx
    )
```

Description

This function frees up any resources tied to the hash context.

Returns

N/A

Errno

N/A

See also

secHash

16. SECHMAC

Name

secHmac - VxWorks HMAC implementation

Layer

SECURITY_MISC_SEC_HASH

Routines

secHmacInit( ) - Initialize a **SEC_HMAC_CTX** **secHmacUpdate( )** - add data to an HMAC calcuation **secHmacFinal( )** - Finalize an HMAC calculation **secHmac( )** - Calculate an HMAC

Description

HMAC is a key MAC (Message Authentication Code), used for message authentication. The HMAC API can be used to calculate an HMAC on a consecutive memory area. If the memory is divided into multiple chunks secHmacInit, secHmacUpdate and secHmacUpdate must be used.

Include files

[**secHmac.h**]

Name

secHmacInit( ) - Initialize a **SEC_HMAC_CTX**

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHmacInit
    (
    SEC_HMAC_CTX*            ctx,
    const unsigned char*     key,
    int                      keyLen,
    const SEC_HASH_TEMPLATE* hashTemplate)
```

Description

Initialize a **SEC_HMAC_CTX**. Must be called before secHmacUpdate and secHmacFinal can be called.

Returns

OK if successful, **ERROR** otherwise

Errno

Not Available

See also

secHmac

Name

secHmacUpdate( ) - add data to an HMAC calcuation

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHmacUpdate
    (
    SEC_HMAC_CTX*        ctx,
    const unsigned char* data,
    unsigned int         dataLen
    )
```

Description

secHmacUpdate can be called repeatedly in order to add multiple chunks of data to an HMAC calculation.

Returns

OK if successful, **ERROR** otherwise.

Errno

Not Available

See also

secHmac

Name

secHmacFinal( ) - Finalize an HMAC calculation

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHmacFinal
    (
    SEC_HMAC_CTX*  ctx,
    unsigned char* hmac,
    unsigned int*  hmacLen
    )
```

Description

Finalize an HMAC calculation. hmacLen must be at least as long as the hash output length.

Returns

OK if successful, **ERROR** otherwise

Errno

Not Available

See also

secHmac

Name

secHmac( ) - Calculate an HMAC

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secHmac
    (
    const unsigned char*     key,
    int                      keyLen,
    unsigned char*           data,
    unsigned int             dataLen,
    unsigned char*           hmac,
    unsigned int*            hmacLen,
    const SEC_HASH_TEMPLATE* hashTemplate)
```

Description

This function calculates an HMAC on a consecutive memory area

Returns

OK if succesful, **ERROR** otherwise

Errno

Not Available

See also

secHmac

17. SECKEYSTORELIB

Name

secKeyStoreLib - VxWorks Key Store

Layer

SECURITY_MISC_SEC_CRYPTO

Routines

secKeyStoreGet( ) - Get a key pair from the key store **secKeyStoreImport( )** - import a key pair into the key store **secKeyStoreX509Get( )** - Get a key pair matching a X509 certificate **secKeyStoreDel( )** - Delete a key pair from the key store

Description

The key store is used to store public key pairs, key pairs used in SSL connections. A typical usage would be an application that listens to incoming SSL connections. This application then needs to have an X509 certificate and a matching private/public key pair. The X509 certification does not need to be protected against unauthorized reading. It can simply be stored in the file system. The key pair, however, is sensitive and should be stored in the key store. First, the key pair must be imported into the key store. This is typically done by a system administrator, e.g. by using the keystore shell command. The system administrator either generates a key pair on the target, or transfers a key pair to the target. Assuming the key is in the file 'mySslKey.pem'. The sysadmin then issues the following command: keyStore import mySslKey mySslKey.pem Now the key pair in mySslKey.pem have been imported into the key store and given the id 'mySslKey'. The sysadmin now deletes mySslKey.pem from the file system. In runtime the application fetches the SSL key from the key store in the following way

```
EVP_PKEY* sslKey;

sslKey = keyStoreGet("mySslKey");
if (!sslKey)
{
    /* error handling */
}
/* use the sslKey as appropriate for your application */


/* Once done with the key, free it */
EVP_PKEY_free(sslKey);
```

The key pairs are stored encrypted in files. A passwords is used to generate the encryption key. The password is called a key-encrypting password. A handler must be configured by means of the **SELECT_SEC_SECRET_KEPW_HANDLER** CDF selection. Note that the key-encrypting password from the VxWorks Vault is used for the Key Store as well.

Include files

[**secKeyStore.h**]

Name

secKeyStoreGet( ) - Get a key pair from the key store

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
EVP_PKEY* secKeyStoreGet
    (
    const char * keyId
    )
```

Description

Fetches a key pair from the key store.

Returns

Pointer to the key pair with the specified key id, or **NULL** if no such key id exist or if an error occurred.

Errno

ENOENT

The key id does not exist

EINVAL

Invalid arguments

ENODEV

The TPM2 OpenSSL engine does not exist.

See also

secKeyStoreLib

Name

secKeyStoreImport( ) - import a key pair into the key store

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
int secKeyStoreImport
    (
    const char * keyId,
    EVP_PKEY *   key
    )
```

Description

This function imports a key pair into the key store and assigns keyId as the key id.

Returns

OK on success, **ERROR** if the key pair could not be imported

Errno

EEXISTS

There is an existing key in the key store with the same key id.

EINVAL

Invalid arguments

See also

secKeyStoreLib

Name

secKeyStoreX509Get( ) - Get a key pair matching a X509 certificate

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
EVP_PKEY* secKeyStoreX509Get
    (
    X509* x509,
    char* keyId,
    int*  keyIdLen
    )
```

Description

Get a key pair matching a X509 certificate. The x509 argument must be set, keyId and keyIdLen are optional and can be set to **NULL**. If keyId and keyIdLen are set they will be updated with the key id of key pair matching the x509 certificate.

Returns

Pointer to the key pair matching a X509 certificate, or **NULL** in case of error

Errno

EINVAL

Invalid arguments

ENOENT

No key pair matching the X509 certificate found.

See also

secKeyStoreLib

Name

secKeyStoreDel( ) - Delete a key pair from the key store

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
int secKeyStoreDel
    (
    const char * id
    )
```

Description

Delete a key pair from the key store

Returns

OK if the key pair was successfully deleted, **ERROR** otherwise

Errno

EINVAL

Invalid arguments

ENOENT

No key pair found with this id

See also

secKeyStoreLib

18. SECPBKDF2

Name

secPbkdf2 - PKCS5 Password-based key derivation function 2.0, RFC 2898

Layer

SECURITY_MISC_SEC_HASH

Routines

secPbkdf2Hmac( ) - PKCS5 Password-based key derivation function 2.0

Description

This module implements PKCS5 Password-based key derivation function 2.0, RFC 2898

Include files

[**secPbkdf2.h**]

Name

secPbkdf2Hmac( ) - PKCS5 Password-based key derivation function 2.0

Layer

SECURITY_MISC_SEC_HASH

Synopsis

```
STATUS secPbkdf2Hmac
    (
    const char              *pw,
    int                     pwLen,
    const unsigned char     *salt,
    int                     saltLen,
    int                     c,
    const SEC_HASH_TEMPLATE *hash,
    int                     keyLen,
    unsigned char           *key
    )
```

Description

PKCS5 Password-based key derivation function 2.0, RFC 2898

Arguments

pw

Password used to to generate keying material

pwLen

Length of the pw argument

salt

Pointer to salt

saltLen

Length of salt argument

c

Iteration count

hash

template for the hash function to use for the HMAC operations. This is typically the return value of a secHashalgTempateGet function, e.g. **secHashSha1TemplateGet( )**

Returns

OK on success, **ERROR** otherwise

Errno

Not Available

See also

secPbkdf2

19. SECSECRETLIB

Name

secSecretLib - VxWorks Secrets Repository

Layer

SECURITY_MISC_SEC_CRYPTO

Routines

secVaultSetKepCb( ) - set the KEP getting routine to secure vault **secVaultGetKepCb( )** - get the pointer of the key-encrypting password (KEP) **secVaultKepBackendSet( )** - register the key-encrypting password (KEP) backend **secSecretKepGet( )** - get the key-encrypting password (KEP) from the backend **secSecretImport( )** - import a secret into the secrets repository **secSecretGet( )** - get a secret from the secrets repository **secSecretDel( )** - delete a secret from the secrets repository

Description

The secrets repository is used to store any buffer that holds sensitive information, for example, symmetric encryption keys and passwords that must be available in clear. Note that passwords that are used to validate logins should be stored in the user management module. The user management module stores passwords in hashed format.

A secret that is used by an application must first be imported into the VxWorks secrets repository. This is typically done by a system administrator using the following command: secret import mySecret The administrator is prompted for the secret. Thereafter, the secret is stored in the VxWorks secrets repository under the id **mySecret**.

At run time, the application fetches the secret from the VxWorks secrets repository as follows:

```
    char secret[32];
    int len = sizeof(secret);

    if (secSecretGet("mySecret", secret, &len) != OK
        {
        /* error handling */
        }

    /*  Use the secret as appropriate for the application */

    /* Overwrite secret when it is no longer needed */
    memset(mySecret, 0, sizeof(mySecret));
```

The buffers are stored encrypted in files. A password is used to generate the encryption key. The password is called a key-encrypting password. A handler must be configured by means of the **SELECT_SEC_SECRET_KEP_HANDLER** CDF selection.

Include files

[**secSecret.h**]

Name

secVaultSetKepCb( ) - set the KEP getting routine to secure vault

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
int secVaultSetKepCb
    (
    SEC_VAULT_KEP_CB callback
    )
```

Description

This function sets the KEP getting function with the vault. This API is deprecated.

Returns

OK, if successful, otherwise **ERROR**

Errno

N/A

See also

secSecretLib

Name

secVaultGetKepCb( ) - get the pointer of the key-encrypting password (KEP)

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
SEC_VAULT_KEP_CB secVaultGetKepCb (void)
```

Description

getting function

This function gets the pointer of the KEP getting function. This API is deprecated.

Returns

the pointer of the KEP getting function if the KEP backend exists, otherwise **NULL**

Errno

N/A

See also

secSecretLib

Name

secVaultKepBackendSet( ) - register the key-encrypting password (KEP) backend

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secVaultKepBackendSet
    (
    SEC_VAULT_KEP_BACKEND * pBackend
    )
```

Description

with the vault

This function registers the KEP backend with the vault.

Arguments

pBackend

It contains the information of the KEP backend. For detail, please reference the structure - **SEC_VAULT_KEP_BACKEND**.

Returns

OK, if successful, otherwise **ERROR**

Errno

N/A

See also

secSecretLib

Name

secSecretKepGet( ) - get the key-encrypting password (KEP) from the backend

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secSecretKepGet
    (
    char * buf,
    int *  len
    )
```

Description

This function gets the KEP from the backend.

Arguments

buf

The buffer for storing the key-encrypting password.

len

This argument is used to indicate the length of the buffer as the input parameter. It is also used to return the actual length of the KEP as the output parameter. Therefore, the value is changed by the backend.

Returns

OK, if successful, otherwise **ERROR**

Errno

ENXIO

Fetching the KEP from the backend failed.

EINVAL

Invalid arguments.

ENODEV

The KEP backend does not exist.

See also

secSecretLib

Name

secSecretImport( ) - import a secret into the secrets repository

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secSecretImport
    (
    const char * id,
    const char * secret,
    int          len
    )
```

Description

This function imports a secret into the secrets repository and assigns it an identity, **id**.

Returns

OK, or **ERROR** if it fails to import successfully

Errno

EEXIST

There is an existing secret in the secrets repository with the same ID.

EINVAL

Invalid arguments.

ENXIO

Fetching the KEP from the backend failed.

ENODEV

The KEP backend does not exist.

See also

secSecretLib

Name

secSecretGet( ) - get a secret from the secrets repository

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
STATUS secSecretGet
    (
    const char * id,
    char *       secret,
    int *        secretLen
    )
```

Description

This function gets a secret from the secrets repository.

Returns

OK, if retrieved successfully, otherwise **ERROR**

Errno

ENOENT

No secret exists with this ID.

EINVAL

Invalid arguments.

ENXIO

Fetching the KEP from the backend failed.

ENODEV

The KEP backend does not exist.

See also

secSecretLib

Name

secSecretDel( ) - delete a secret from the secrets repository

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
int secSecretDel
    (
    const char * id
    )
```

Description

This function deletes a secret from the secrets repository.

Returns

OK, if successful, otherwise **ERROR**

Errno

EINVAL

Invalid arguments.

See also

secSecretLib

20. SECTRUSTSTORELIB

Name

secTrustStoreLib - Trust Store Lib

Layer

SECURITY_MISC_SEC_CRYPTO

Routines

secTrustStorePubKeyValidateProviderSet( ) - install a trust store provider

Description

The trust store provides an API that enables you to check on whether a public key is trusted or not. This can be used by, for example, the secure ELF loader in order to determine if a public key used to verify a signature on ELF files is trusted. The library also provides the ability to install different providers for the trust check.

Include files

[**secTrustStore.h**]

Name

secTrustStorePubKeyValidateProviderSet( ) - install a trust store provider

Layer

SECURITY_MISC_SEC_CRYPTO

Synopsis

```
void secTrustStorePubKeyValidateProviderSet
    (
    SEC_TRUST_STORE_PUBKEY_VALIDATE_CB provider
    )
```

Description

This function install a trust store provider.

Returns

N/A

Errno

N/A

See also

secTrustStoreLib

21. UDBMGR

Name

udbMgr - VxWorks User Database Manager

Layer

USER_MANAGEMENT

Routines

udbBase64Encode( ) - base64-encode a message **udbBase64Decode( )** - decode a Base64-encoded message **udbTamperingPolicySet( )** - set the tampering policy handler **udbTamperingPolicyGet( )** - get the tampering policy handler's address **udbInfoGet( )** - get information about the user database **udbDevCheck( )** - check the device for the user database **udbReset( )** - reset the user database **udbInfoShow( )** - show information about the user database

Description

This library provides the functions that handle the VxWorks User Database (UDB). The user database stores information about the users allowed on the VxWorks system. This library provides few API as it is typically used in conjunction with the User Identification facility (**userIdentLib**) which provides API for adding and removing users, as well as for updating user records.

The user database is typically created once. Then content - user records - is added to or removed from it. Creation of the user database is done automatically at system's initialization time, based on the VxWorks User Database's configuration parameters (see below) but it may happen that the user database has to be recreated at a later point. In order to avoid mistakenly erasing the user database's content, its existence can be checked via the **udbInfoGet( )** API. The database can then be created via the **udbReset( )** API.

```
    /* Check whether the VxWorks user database has already been created */

    if (udbInfoGet (NULL, NULL, NULL, NULL) == ERROR)
        {
        /* The user database does not exist; let's create it */

        if (udbReset() != OK)
            {
            <print an error message>
            ...
            }
        }
```

A show routine, **udbInfoShow( )** can be used to display the database's storage path, current number of user records, version number and whether or not the Message Authentication Code (MAC) computing is the default one or is provided by a user-defined function.

Once the user database is created it can be populated (see the User Identification facility). Note that if the user database is empty but the shell's login phase is activated (this is the default behavior), the first user to log-in will be considered the system initial's user and will _not_ be challenged. Instead the initial user is prompted for a login name and a password that will then be added to the user database. If this behavior is not desired the user database must be populated with at least one user record at setup time.

A user database created by a previous version of the user database manager may have to be converted when re-used on a system running an updated version of the manager. To the extent that this is possible the conversion is done automatically. Default values are then used for the new fields as described below. The conversion cannot be done automatically when the user

database is on a read-only filesystem or not enough room is left on the filesystem. In those situations the system's configuration must be changed so as to allow for the conversion to proceed. The conversion cannot be done automatically when the user database appears damaged (invalid information or user records) or its detected version is unsupported. In those cases the user database will have to be entirely re-created.

Conversions currently supported:

Version 1 to version 2

Addition of a group ID field. Default value, 100.

A user-provider tampering policy handler can be set up either via the CDL parameter **UDB_CUSTOM_TAMPERING_POLICY_RTN** or via the **udbTamperingPolicySet( )** API (see the Configuration section below for more details). This handler will be called whenever the user database manager detects that the database may have been tampered with. Using the CDL parameter to set this handler ensures that it is available as early as possible in the lifetime of the system. A default handler exists that blocks user authentication and logs tampering detection events in the ED&R log, if this feature is configured in the system:

```
void defaultUdbTamperingHdlr
    (
    UDB_EVENT_INFO info    /* information about tampering event */
    )
    {
    char msg[100];

    userAuthBlock();
    loginAuthBlockedMsgSet ("<A suitable message for loginLib to display>");
    (void) snprintf (msg, 100,
                     "User Database: tampering event type %d detected for "
                     "record at offset %ld!\n",
                     info.eventType, info.recordOffset);
    EDR_INJECT (EDR_SEVERITY_WARNING | EDR_FACILITY_KERNEL,
                __FILE__, __LINE__, NULL, NULL, NULL, msg);
    }
```

The tampering policy handler is passed a **UDB_EVENT_INFO** structure which holds information that may be useful for coping with the tampering event.

A user-provider tampering policy handler can also reset the user database to its initial (empty) state, however if user authentication is not blocked the first next user to log-in will automatically be considered the initial user and will be prompted for a login name and a password to add to the user database.

Each entry of the user database is validated through the computation of a message authentication code (MAC). By default the user database manager uses the SHA-256 cryptographic hash to generate hash-based message authentication codes (HMAC).

Requirements

The user database manager requires that the user database file be stored on a file system supporting formatted I/O operations (**fopen( )**, **fread( )**, **fwrite( )**, etc.) as well as file truncation (**ftruncate( )**). The user database file should be stored on a storage unit attached to the target (as opposed to accessed via the network).

If no storage device is available it is still possible to store the user database file on ROMFS but it can neither be created there at runtime nor be updated (addition and removal of users as well as password changes). The latter constraint is a clear security risk in case a user's password has been compromised, as it cannot be changed without regenerating the VxWorks image. Therefore storing the user database in ROMFS is strongly advised against.

Nevertheless, in order to store the user database file on ROMFS it must first be created on a writable file system, for instance a RAM disk formatted with HRFS, and populated with all the user information that will be needed. Then the file can be copied in

a "romfs" directory in a VxWorks image project (VIP) and the **UDB_STORAGE_PATH** CDL parameter be set to reflect the path and name of the database file (see the configuration section below). When a VxWorks image will be generated in this VIP it will automatically use the user database stored on ROMFS.

A random key at least 256 bytes long must be passed to the user database manager's initialization routine otherwise the user database will not be safe enough. This key is used by the default cryptographic hash function, or by the custom MAC computing function, to compute the validation value associated to each record in the database. It is strongly suggested never to use the same key across several devices. To be effective the key must be specific to a device. Note that changing the key for a device where a user database already exists will lead to that database's content to be unusable.

Configuration

The **UDB_STORAGE_PATH** CDL parameter indicates where the user database file is to be found or created. It holds a strings corresponding to the absolute path to the file, including the file name.

The **UDB_HASH_KEY** CDL parameter holds the random hash key needed to compute the MAC values. It consists in a string made of at least 256 random binary values in the form **\xdd** where **dd** is a byte value in hexadecimal format.

The **UDB_CUSTOM_TAMPERING_POLICY_RTN** parameter holds the name (interpreted as an address) of a user-provided handler that will be called whenever the user database manager detects that tampering occurred on a record in the user database. Two options exist for setting this parameter: either it is set with the name of the **udbCustomPolicyHandler( )** routine, or it is set with the name of a user-defined routine hosted by a user-provided file.

In the first use case the inclusion of the **INCLUDE_UDB_CUSTOM_TAMPERING_POLICY** component will automatically add the user configlette **udbCustomPolicyHandler.c** to the VxWorks image project (VIP) and the parameter **UDB_CUSTOM_TAMPERING_POLICY_RTN** will automatically be set with the name "udbCustomPolicyHandler". The user-defined code must then be added to the routine **udbCustomPolicyHandler( )** defined in this file.

In the second use case more configuration steps are needed: 1) the source file holding the custom tampering policy handler must be added to the VxWorks image project (VIP):

```
% vxprj file add <path to the custom tampering policy handler's source file>
```

And, 2) a CDF file declaring a user-defined component must be added to the VIP so that the prototype and header files needed by the custom tampering policy handler are known to the VIP build infrastructure:

```
Component INCLUDE_CUSTOM_TAMPERING_POLICY_HDLR {
        HDR_FILES       udbMgr.h
        PROTOTYPE       extern void <name of routine> (UDB_EVENT_INFO);
        _REQUIRED       INCLUDE_USER_DATABASE
}
```

The name of the component itself is user-defined but must not collide with an already existing component name. The name of the CDF file is user-defined too but must have the **cdf** extension for it to be taken into account. This user-defined component is then included in the VIP as follows:

```
% vxprj component add INCLUDE_CUSTOM_TAMPERING_POLICY_HDLR
```

Finally the name of the custom tampering policy handler must be set via the **UDB_CUSTOM_TAMPERING_POLICY_RTN** parameter:

```
% vxprj parameter set <name of routine>
```

Include files

none

Name

udbBase64Encode( ) - base64-encode a message

Layer

USER_MANAGEMENT

Synopsis

```
size_t udbBase64Encode
    (
    char * message,   /* message to encode */
    size_t msgLen,    /* length of the message to encode */
    char * b64Msg     /* where to store the encoded message */
    )
```

Description

This routine encodes a message using the Base64 encoding scheme. The b64Msg parameter points to a memory area allocated by the caller. This memory area must be big enough to hold the full Base64-encoded message. The minimal size of this memory area can be determined by calling this routine with the message to encode and the address of a large memory buffer. The return value of the routine will be set to the minimal size of the buffer needed to encode the message.

Returns

the minimal size of the buffer needed to encode the message.

Errnos

N/A

See also

udbMgr

Name

udbBase64Decode( ) - decode a Base64-encoded message

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbBase64Decode
    (
    char *   b64Msg,      /* encoded message to decode */
```

```
    size_t   b64MsgLen,   /* length of encoded message */
    char *   message,     /* where to store decoded message */
    size_t * pMsgLen      /* where to store the length of the decoded msg */
    )
```

Description

This routine decodes a message encoded by the **udbBase64Encode( )** API. The message parameter points to a memory area allocated by the caller. This memory area must be big enough to hold the decoded message.

Returns

OK or **ERROR**

Errnos

N/A

See also

udbMgr

Name

udbTamperingPolicySet( ) - set the tampering policy handler

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbTamperingPolicySet
    (
    UDB_TAMPER_POLICY_RTN callMe   /* pointer to a user-provided routine */
    )
```

Description

This function registers a user-defined routine to call immediately when tampering has been detected.

The prototype of the user-defined routine must be as follows: void routineName (**UDB_EVENT_INFO**);

Returns

OK, or **ERROR**

Errnos

N/A

See also

udbMgr

Name

udbTamperingPolicyGet( ) - get the tampering policy handler's address

Layer

USER_MANAGEMENT

Synopsis

```
UDB_TAMPER_POLICY_RTN udbTamperingPolicyGet (void)
```

Description

This function returns the address of the currently registered tampering policy handler. Note that if no user-provided handler has been set, the system will set a default handler which address is then returned.

Returns

the address of the current handler.

Errnos

N/A

See also

udbMgr

Name

udbInfoGet( ) - get information about the user database

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbInfoGet
    (
    char *  udbPath,         /* where to store the database's path */
    int *   pUdbVersion,     /* where to store the database's version number */
    int *   pUserRecordNb,   /* where to store the number of user records */
    uid_t * pHighestUid      /* where to store the highest UID used in the DB */
    )
```

Description

This routine provides the following information about the user database: - path to the storage of the database - version number of the database - the number of user records in the database, - the highest UID currently applied to a user record in the database.

Information is provided only when the relevant parameter is non-null. The buffer for the database path must be at least **PATH_MAX** in length.

Note that the information record is not accounted for in the record number (i.e. an empty database only holds the information records which indicates that there are zero user records in the database).

Returns

ERROR if an issue happened, **OK** otherwise.

Errnos

N/A

See also

udbMgr

Name

udbDevCheck( ) - check the device for the user database

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbDevCheck (void)
```

Description

This routine checks if the device for saving the user database is ready. It is possible that the device is not ready when want to initialize or use the user database manager.

Returns

OK if the device is ready, **ERROR** otherwise.

Errnos

N/A

See also

udbMgr

Name

udbReset( ) - reset the user database

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbReset (void)
```

Description

This routine resets the user database by reseting the content of the database info record as follows: the database's total size is reset to the size of the info record only and the number of user records is reset to zero.

Once the UDB information record is reset the existing user records are zeroed out and thus definitively lost.

If the storage for the user database does not exist, this routine will create it.

Returns

OK of the database has been reset, **ERROR** otherwise.

Errnos

N/A

See also

udbMgr

Name

udbInfoShow( ) - show information about the user database

Layer

USER_MANAGEMENT

Synopsis

```
STATUS udbInfoShow (void)
```

Description

This routine displays some information about the user database: storage path, version and current number of records.

Returns

OK, or **ERROR** if a problem occurred.

Errno

N/A

See also

udbMgr

22. USERAUTH

Name

userAuth - VxWorks user authentication

Layer

USER_MANAGEMENT

Routines

userCtxAuthenticate( ) - authenticate a user by name and password **userAuthenticate( )** - authenticate a user by name and password **userLogout( )** - logout user **userCtxAuthHookSet( )** - Register the user authentication function hook **userLogoutHookSet( )** - Register the user logout function hook **loginCtxInit( )** - Initialize login context **loginCtxInterfaceSet( )** - Set the login interface ID **loginCtxInterfaceGet( )** - Get the login interface ID **loginCtxFdSet( )** - Set the login file descriptor **loginCtxFdGet( )** - Get the login file descriptor **pwValidGet( )** - Get password valid status **userDefinedGet( )** - Get user defined status

Description

The file provides a unified authentication API which can invoke different authentication back-end, for example, local user authentication, ad/ldap user authenticaiton and customized user authentication.

Include files

Name

userCtxAuthenticate( ) - authenticate a user by name and password

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userCtxAuthenticate
    (
    const char * loginName,       /* login name for the user to authenticate */
    const char * clearPassword,   /* cleartext password of user to authenticate */
    uid_t *      pUserId,         /* where to store the user's user ID */
    gid_t *      pGroupId,        /* where to store the user's group ID */
    LOGIN_CTX*   ctx              /* Object for login context information */
    )
```

Description

This routine authenticates a user given a name and password. Additional login- related information can be provided in the **LOGIN_CTX** argument. The **LOGIN_CTX** argument must be initialized by calling loginCtxInit and thereafter it is populated by calls to e.g. loginCtxInterfaceSet, loginCtxFdSet, etc. If the user is known to the system and the pUserId parameter is not **NULL**, the user's user ID is provided to the caller. Similarly, if the pGroupId parameter is not **NULL** the user's primary group ID is provided to the caller.

Returns

OK, or **ERROR** if the user is not found by name or the provided password does not match the record.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record, or a login policy rejected the authentication.

S_userIdentLib_USER_DISABLED

The user account is disabled

See also

userAuth

Name

userAuthenticate( ) - authenticate a user by name and password

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userAuthenticate
    (
    const char * loginName,       /* login name for the user to authenticate */
    const char * clearPassword,   /* cleartext password of user to authenticate */
    uid_t *      pUserId,         /* where to store the user's user ID */
    gid_t *      pGroupId         /* where to store the user's group ID */
    )
```

Description

This routine authenticates a user given a name and password. If the user is known to the system and the pUserId parameter is not **NULL**, the user's user ID is provided to the caller. Similarly, if the pGroupId parameter is not **NULL** the user's primary group ID is provided to the caller.

Returns

OK, or **ERROR** if the user is not found by name or the provided password does not match the record.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record, or a login policy rejected the authentication.

S_userIdentLib_USER_DISABLED

The user account is disabled

See also

userAuth

Name

userLogout( ) - logout user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLogout
    (
    const char * loginName,   /* login name for the user to log out */
    LOGIN_CTX*   ctx          /* Container for additional info */
    )
```

Description

Logs user out from the system

Returns

OK or **ERROR** upon failure

Errno

Not Available

See also

userAuth

Name

userCtxAuthHookSet( ) - Register the user authentication function hook

Layer

USER_MANAGEMENT

Synopsis

```
void userCtxAuthHookSet
    (
```

```
    USER_CTX_AUTH_HOOK userCtxAuthHook
    )
```

Description

Register a user authentication function to the authentication function hook.

Returns

N/A

Errno

N/A

See also

userAuth

Name

userLogoutHookSet( ) - Register the user logout function hook

Layer

USER_MANAGEMENT

Synopsis

```
void userLogoutHookSet
    (
    USER_LOGOUT_HOOK userLogoutHook
    )
```

Description

Register a user logout function to the logout function hook.

Returns

N/A

Errno

N/A

See also

userAuth

Name

loginCtxInit( ) - Initialize login context

Layer

USER_MANAGEMENT

Synopsis

```
void loginCtxInit
    (
    LOGIN_CTX* ctx
    )
```

Description

Initialize login context. This function must be called on a **LOGIN_CTX** struct before it can be used as an argument to function calls.

Returns

N/A

Errno

N/A

See also

userAuth

Name

loginCtxInterfaceSet( ) - Set the login interface ID

Layer

USER_MANAGEMENT

Synopsis

```
void loginCtxInterfaceSet
    (
    LOGIN_CTX* ctx,
    int        interface
    )
```

Description

Set the login interface ID. Common values are

LOGIN_INTERFACE_UNDEFINED
LOGIN_INTERFACE_TELNET
LOGIN_INTERFACE_RLOGIN
LOGIN_INTERFACE_SSH
LOGIN_INTERFACE_FTP
LOGIN_INTERFACE_CONSOLE

Returns

N/A

Errno

N/A

See also

userAuth

Name

loginCtxInterfaceGet( ) - Get the login interface ID

Layer

USER_MANAGEMENT

Synopsis

```
int loginCtxInterfaceGet
    (
    LOGIN_CTX* ctx
    )
```

Description

Get the login interface

Returns

The login interface ID

Errno

N/A

See also

userAuth

Name

loginCtxFdSet( ) - Set the login file descriptor

Layer

USER_MANAGEMENT

Synopsis

```
void loginCtxFdSet
    (
    LOGIN_CTX* ctx,
    int        fd
    )
```

Description

Set the file descriptor of the connection an authentication request was received on.

Returns

N/A

Errno

N/A

See also

userAuth

Name

loginCtxFdGet( ) - Get the login file descriptor

Layer

USER_MANAGEMENT

Synopsis

```
int loginCtxFdGet
    (
    LOGIN_CTX* ctx
    )
```

Description

Get the file descriptor of the connection an authentication request was received on.

Returns

File descriptor

Errno

N/A

See also

userAuth

Name

pwValidGet( ) - Get password valid status

Layer

USER_MANAGEMENT

Synopsis

```
BOOL pwValidGet
    (
    const LOGIN_CTX* ctx   /* login context */
    )
```

Description

This function makes it possible to check if a password is valid. It is used in the implementation of a login-policy-enforcer callback.

Returns

TRUE if the password is valid, **FALSE** otherwise

Errno

N/A

See also

userAuth

Name

userDefinedGet( ) - Get user defined status

Layer

USER_MANAGEMENT

Synopsis

```
BOOL userDefinedGet
    (
    const LOGIN_CTX* ctx   /* login context */
    )
```

Description

This function makes it possible to check whether a user account is defined. It is used in the implementation of a login-policy-enforcer callback.

Returns

TRUE if the password is valid, **FALSE** otherwise*

Errno

N/A

See also

userAuth

23. USERIDENTLIB

Name

userIdentLib - VxWorks user identification library

Layer

USER_MANAGEMENT

Routines

userAdd( ) - create a new user account **userDelete( )** - delete an existing user account **userPasswordUpdate( )** - update the password for an existing user **userPasswordReset( )** - Reset the password for an existing user **userGroupUpdate( )** - update the group ID for an existing user **userLastLoginTimeGet( )** - Get time of last login for a user **userLastLoginTimeSet( )** - Set last login time of a user **userLastLoginUpdate( )** - Set last login time for **loginName** to the current time **userLastPwChangeTimeSet( )** - Get the lastPwChange time for **loginName** **userLastPwChangeTimeSet( )** - Set the lastPwChange time for **loginName** **userNameUpdate( )** - update the login name for an existing user **userEnableSet( )** - Set the enable flag for a user in the user DB **userEnableGet( )** - Get the enable flag for a user from the user DB **userExpireDateSet( )** - Set expire date of a user **getpwuid( )** - search the user database for a user ID (POSIX) **getpwnam( )** - search the user database for a user name (POSIX) **getpwuid_r( )** - search the user database for a user ID (POSIX) **getpwnam_r( )** - search the user database for a user name (POSIX)

Description

This library provides the facility needed to manage the native user concept on VxWorks. It provides API for adding and deleting users to and from the system, authenticating users based on the information stored in the user database, updating the user's password, group ID and login name. It also provides API to block - and unblock - the user authentication should this be necessary, for instance to temporarily prevent users to log in.

The VxWorks modules that perform authentication, e.g. telnet and SSH servers, use **userIdentLib**. A customer that has an application-specific module that needs to perform authentication can use **userIdentLib** to do so. The required code to do so for a proprietary telnet server would look something like this:

```
LOGIN_CTX loginCtx;

loginCtxInit(&loginCtx);
loginCtxSetInterface(&loginCtx, LOGIN_INTERFACE_TELNET);
loginCtxSetFd(&loginCtx, telnetClientSock);
if (userCtxAuthenticate(username, passwd, NULL, NULL, &loginCtx) != OK)
    {
     Error handling
    }
```

In the current version not all APIs are supported from RTPs. See the RTP-specific reference documentation for details.

This library also provides the POSIX **getpwuid( )**, **getpwnam( )**, **getpwuid_r( )** and **getpwnam_r( )** API. Note that the **getpwuid( )** and **getpwnam( )** API are not thread-safe.

Configuration

The user identification feature is included when the **INCLUDE_USER_IDENTIFICATION** component is added to a vxWorks image project (VIP). Note that, for convenience, including the **INCLUDE_SECURITY** component will automatically add the user identification feature and all its dependencies.

Include files

none

Name

userAdd( ) - create a new user account

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userAdd
    (
    const char * loginName,       /* login name for the new user */
    const char * clearPassword,   /* cleartext password of user */
    const uid_t  userId,          /* user's user ID */
    const gid_t  groupId          /* user's primary group ID */
    )
```

Description

This routine creates a new user account for the system. The new user is recorded in the user database.

The loginName string must not be greater than **MAX_LOGIN_NAME_LEN** (80 characters). The clearPassword string must not be greater than BUFSIZ (1024 characters).

When the userId parameter is set to the value 0 the system is instructed to assign a default, unique, user ID. While it is preferable to let the system assign user IDs, it is possible to request the assignment of a specific user ID to a newly created account. When a user's ID is specified (i.e. userId parameter set with a value different from 0) it also sets the starting point for the next automatic assignment of a user ID. For instance if the first created user account is associated with the ID **100**, the next user ID automatically assigned by the system will be **101**. When the largest user ID value (65535) has been used, on the next automatic assignment the system will attempt at finding user IDs unassigned in the user database and assigns the smallest found to the new user. If none can be found an error status is returned.

Note: automatically assigned user IDs start with the value 2; the user ID of value 1 is reserved to the root user and cannot be used otherwise. There is no user ID of value zero.

When the groupId parameter is set to the value 0 the system will apply the default primary group ID value set via the **DEFAULT_PRIMARY_GROUP_ID** CDL parameter.

Note: the group ID of value 1 is reserved to the root user and cannot be used otherwise. There is no group ID of value zero.

No privilege verifications are performed by this routine.

Returns

OK, or **ERROR** if the login name is invalid or the user ID is already used.

Errno

Possible errnos returned from this routine are:

S_userIdentLib_INVALID_USER_NAME

The loginName parameter is null, points to an empty string or points to a string that is longer than **MAX_LOGIN_NAME_LEN**.

S_userIdentLib_USER_ALREADY_EXISTS

The user with login name loginName already has an account.

S_userIdentLib_USER_ID_ALREADY_ASSIGNED

The user ID userId is already used for another user account.

S_userIdentLib_NO_MORE_USER_ID

There is no free user ID to use; the user table is full.

S_userIdentLib_ROOT_IDS_MISMATCH

Either one of the user or group ID is not that of the root user when the other one is.

See also

userIdentLib

Name

userDelete( ) - delete an existing user account

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userDelete
    (
    const char * loginName,   /* name of existing user to be deleted */
    const uid_t  userId       /* ID of existing user to be deleted */
    )
```

Description

This routine removes an existing user account from the system. The existing user is removed from the user database.

The user account is found based either on the user's login name, if non-**NULL**, or the user ID, if greater than zero. If both are provided, the user ID takes precedence (i.e. the user name is ignored).

No privilege verifications are performed by this routine.

Returns

OK, or **ERROR** if the login name is not valid or the user account cannot be deleted.

Errno

Possible errnos returned from this routine are:

S_userIdentLib_INVALID_USER_NAME

The loginName parameter points to an empty string.

S_userIdentLib_INVALID_USER_ID

The userId parameter holds zero while the loginName parameter is **NULL**.

See also

userIdentLib

Name

userPasswordUpdate( ) - update the password for an existing user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userPasswordUpdate
    (
    const char * loginName,          /* login name of user to update */
    const char * oldClearPassword,   /* user's old password in cleartext */
    const char * newClearPassword    /* user's new password in cleartext */
    )
```

Description

This routine updates the password of the user loginName. The new password is to be passed as cleartext in newClearPassword.

No privilege verifications are performed by this routine but the old password must be passed and must be successfully verified before the password is changed.

The old password is to be passed as cleartext in oldClearPassword.

Returns

OK, or **ERROR** if the password could not be updated.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record.

See also

userIdentLib

Name

userPasswordReset( ) - Reset the password for an existing user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userPasswordReset
    (
    const char * loginName,         /* login name of user to update */
    const char * newClearPassword   /* user's new password in cleartext */
    )
```

Description

This routine Reset the password of the user loginName. The new password is to be passed as cleartext in newClearPassword.

No privilege verifications are performed by this routine.

Returns

OK, or **ERROR** if the password could not be updated.

Errno

S_userIdentLib_AUTHENTICATION_FAILED

The user could not be authenticated: either this user does not exist or the provided password does not match the record.

See also

userIdentLib

Name

userGroupUpdate( ) - update the group ID for an existing user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userGroupUpdate
    (
    const char * loginName,   /* login name of user to update */
    gid_t        newGroupId   /* updated group ID */
    )
```

Description

This routine updates the group ID of the user loginName. The newGroupId parameter must be a value between 1 and 65535. Note that the value 1 may be used only for the root user.

No privilege verifications are performed by this routine.

Returns

OK, or **ERROR** if the group ID could not be updated.

Errno

S_userIdentLib_INVALID_GROUP_ID

The new group ID is not a valid value.

S_userIdentLib_PERMISSION_DENIED

The new group ID is that of the root user but the user is not the root user.

See also

userIdentLib

Name

userLastLoginTimeGet( ) - Get time of last login for a user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLastLoginTimeGet
    (
    const char* loginName,   /* login name of user */
    time_t*     pLoginTime   /* pointer to the time_t variable to return the result in */
    )
```

Description

This function returns the last time **loginName** was successfully logged in.

Returns

OK if pLoginTime has been updated with the last login time, **ERROR** otherwise

Errno

EINVAL

loginName or pLoginTime is **NULL**

See also

userIdentLib

Name

userLastLoginTimeSet( ) - Set last login time of a user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLastLoginTimeSet
    (
    const char* loginName,   /* login name of user to update */
    time_t      loginTime    /* new value for the lastLogin field */
    )
```

Description

Set last login time of a user

Returns

OK if the last login time was updated, **ERROR** otherwise

Errno

S_userIdentLib_INVALID_USER_NAME

loginName is **NULL**

See also

userIdentLib

Name

userLastLoginUpdate( ) - Set last login time for **loginName** to the current time

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLastLoginUpdate
    (
    const char* loginName   /* login name of user to update */
    )
```

Description

This function sets the last login time for **loginName** to the current time.

Returns

OK if successful, **ERROR** otherwise.

Errno

Not Available

See also

userIdentLib

Name

userLastPwChangeTimeSet( ) - Get the lastPwChange time for **loginName**

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLastPwChangeTimeGet
    (
    const char* loginName,      /* login name of user */
    time_t*     pPwchangetime   /* pointer to the time_t variable to return the result in */
    )
```

Description

Get the lastPwChange time for **loginName**

Returns

OK if successful, **ERROR** otherwise

Errno

EINVAL

loginName or pLoginTime is **NULL**

See also

userIdentLib

Name

userLastPwChangeTimeSet( ) - Set the lastPwChange time for **loginName**

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userLastPwChangeTimeSet
    (
    const char* loginName,     /* login name of user to update */
    time_t      pwchangetime   /* new value for the lastPwChange field */
    )
```

Description

Set the lastPwChange time for **loginName**

Returns

OK if successful, **ERROR** otherwise

Errno

Not Available

See also

userIdentLib

Name

userNameUpdate( ) - update the login name for an existing user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userNameUpdate
    (
    const char * loginName,     /* login name of user to update */
    const char * newLoginName   /* new login name of user */
    )
```

Description

This routine updates the login name of the user loginName. The only constraint for the newLoginName parameter is the size of the string.

No privilege verifications are performed by this routine.

Returns

OK, or **ERROR** if the login name could not be updated.

Errno

S_userIdentLib_INVALID_USER_NAME

Either of the loginName or newLoginName parameters holds an invalid user name.

S_userIdentLib_PERMISSION_DENIED

The new group ID is that of the root user but the user is not the root user.

See also

userIdentLib

Name

userEnableSet( ) - Set the enable flag for a user in the user DB

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userEnableSet
    (
    const char* loginName,   /* login name of user to update */
    BOOL        enabled      /* new value for the enabled field */
    )
```

Description

Set the enable flag in the user DB

Returns

OK on success, **ERROR** otherwise

Errno

S_userIdentLib_INVALID_USER_NAME

The loginName argument is **NULL**

See also

userIdentLib

Name

userEnableGet( ) - Get the enable flag for a user from the user DB

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userEnableGet
    (
    const char* loginName,   /* login name of user  */
    BOOL*       enabled      /* pointer to BOOL return variable */
    )
```

Description

Get the enable flag for a user from the user DB

Returns

OK on success, **ERROR** otherwise

Errno

S_userIdentLib_INVALID_USER_NAME

The loginName argument is **NULL**

EINVAL

The enabled argument is **NULL**

See also

userIdentLib

Name

userExpireDateSet( ) - Set expire date of a user

Layer

USER_MANAGEMENT

Synopsis

```
STATUS userExpireDateSet
    (
    const char*  loginName,   /* login name of user to update */
    unsigned int date         /* new value for the expirationDate field */
    )
```

Description

Set expire date of a user

Returns

OK if the expire was updated, **ERROR** otherwise

Errno

S_userIdentLib_INVALID_USER_NAME

loginName is **NULL**

See also

userIdentLib

Name

getpwuid( ) - search the user database for a user ID (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
struct passwd * getpwuid
    (
    uid_t userId   /* UID of user whose information is of interest */
    )
```

Description

This routine searches the user database for entries matching the provided user ID.

Caveat

use of this routine is strongly discouraged as it is not thread-safe and the content of the **passwd** structure and associated storage area may be overwritten by another task. Usage of **getpwuid_r( )** is recommended instead.

Returns

a pointer to a static **passwd** structure if a matching entry is found, a **NULL** pointer otherwise or if an error occurred.

Errno

Not Available

See also

userIdentLib

Name

getpwnam( ) - search the user database for a user name (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
struct passwd * getpwnam
    (
    const char * loginName   /* name of user whose information is of interest */
    )
```

Description

This routine searches the user database for entries matching the provided user name.

Caveat

use of this routine is strongly discouraged as it is not thread-safe and the content of the **passwd** structure and associated storage area may be overwritten by another task. Usage of **getpwnam_r( )** is recommended instead.

Returns

a pointer to a **passwd** structure if a matching entry is found, a **NULL** pointer otherwise or if an error occurred.

Errno

Not Available

See also

userIdentLib

Name

getpwuid_r( ) - search the user database for a user ID (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
int getpwuid_r
    (
    uid_t            userId,    /* UID of user whose information is of interest */
    struct passwd *  pwd,       /* where to store the user information */
    char *           buffer,    /* allocated memory for information */
    size_t           bufsize,   /* size of allocated memory */
    struct passwd ** result     /* where to store the address of the user info */
    )
```

Description

This routine searches the user database for entries matching the provided user ID. If the search is successful, the data structure pointed to by the pwd parameter holds the relevant user information and the address pointed to by the result parameter is set with the address of the pwd structure. If the search is not successful the data structure pointed to by the pwd parameter is not updated and the address pointed to by the result parameter is set to **NULL**.

The buffer parameter should point to a block of allocated memory of at least **MAX_LOGIN_NAME_LEN** in size.

Returns

zero if successful, otherwise an error code indicative of the error.

Errnos

EINVAL

One of the pwd or result parameters is null.

ENOMEM

Not enough memory left to allocate internal data structures.

ERANGE

The buffer provided for storing the user information is too small.

ESRCH

No matching user database entry was found for the provided UID.

See also

userIdentLib

Name

getpwnam_r( ) - search the user database for a user name (POSIX)

Layer

USER_MANAGEMENT

Synopsis

```
int getpwnam_r
    (
    const char *     loginName,   /* name of user whose info is of interest */
    struct passwd *  pwd,         /* where to store the user information */
    char *           buffer,      /* allocated memory for information */
    size_t           bufsize,     /* size of allocated memory */
    struct passwd ** result       /* where to store the address of the user info */
    )
```

Description

This routine searches the user database for entries matching the provided user name. If the search is successful, the data structure pointed to by the pwd parameter holds the relevant user information and the address pointed to by the result parameter is set with the address of the pwd structure. If the search is not successful the data structure pointed to by the pwd parameter is not updated and the address pointed to by the result parameter is set to **NULL**.

The buffer parameter should point to a block of allocated memory of at least **MAX_LOGIN_NAME_LEN** in size.

Returns

zero if successful, otherwise an error code indicative of the error.

Errnos

EINVAL

One of the pwd or result parameters is null.

ENOMEM

Not enough memory left to allocate internal data structures.

ERANGE

The buffer provided for storing the user information is too small.

ESRCH

No matching user database entry was found for the provided UID.

See also

userIdentLib